

Imports ¶

In [1]:

```
!pip install pmdarima  
!pip install git+https://github.com/scikit-learn-contrib/py-earth@v0.2dev
```

Collecting pmdarima

Downloading pmdarima-1.8.2-cp37-cp37m-manylinux1_x86_64.whl (1.5 MB)

|██| 1.5 MB 5.2 MB/s

Collecting statsmodels!=0.12.0,>=0.11

Downloading statsmodels-0.12.2-cp37-cp37m-manylinux1_x86_64.whl (9.5 MB)

|██| 9.5 MB 51.8 MB/s

Requirement already satisfied: pandas>=0.19 in /usr/local/lib/python3.7/dist-packages (from pmdarima) (1.1.5)

Requirement already satisfied: scipy>=1.3.2 in /usr/local/lib/python3.7/dist-packages (from pmdarima) (1.4.1)

Requirement already satisfied: urllib3 in /usr/local/lib/python3.7/dist-packages (from pmdarima) (1.24.3)

Requirement already satisfied: Cython!=0.29.18,>=0.29 in /usr/local/lib/python3.7/dist-packages (from pmdarima) (0.29.24)

Requirement already satisfied: numpy~=1.19.0 in /usr/local/lib/python3.7/dist-packages (from pmdarima) (1.19.5)

Requirement already satisfied: setuptools!=50.0.0,>=38.6.0 in /usr/local/lib/python3.7/dist-packages (from pmdarima) (57.4.0)

Requirement already satisfied: scikit-learn>=0.22 in /usr/local/lib/python3.7/dist-packages (from pmdarima) (0.22.2.post1)

Requirement already satisfied: joblib>=0.11 in /usr/local/lib/python3.7/dist-packages (from pmdarima) (1.0.1)

Requirement already satisfied: python-dateutil>=2.7.3 in /usr/local/lib/python3.7/dist-packages (from pandas>=0.19->pmdarima) (2.8.2)

Requirement already satisfied: pytz>=2017.2 in /usr/local/lib/python3.7/dist-packages (from pandas>=0.19->pmdarima) (2018.9)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.7/dist-packages (from python-dateutil>=2.7.3->pandas>=0.19->pmdarima) (1.15.0)

Requirement already satisfied: patsy>=0.5 in /usr/local/lib/python3.7/dist-packages (from statsmodels!=0.12.0,>=0.11->pmdarima) (0.5.1)

Installing collected packages: statsmodels, pmdarima

Attempting uninstall: statsmodels

Found existing installation: statsmodels 0.10.2

Uninstalling statsmodels-0.10.2:

Successfully uninstalled statsmodels-0.10.2

Successfully installed pmdarima-1.8.2 statsmodels-0.12.2

Collecting git+https://github.com/scikit-learn-contrib/py-earth@v0.2dev

Cloning https://github.com/scikit-learn-contrib/py-earth (to revision v0.2dev) to /tmp/pip-req-build-cbgsavh

Running command git clone -q https://github.com/scikit-learn-contrib/py-earth /tmp/pip-req-build-cbgsavh

Running command git checkout -b v0.2dev --track origin/v0.2dev

Switched to a new branch 'v0.2dev'

Branch 'v0.2dev' set up to track remote branch 'v0.2dev' from 'origin'.

Requirement already satisfied: scipy>=0.16 in /usr/local/lib/python3.7/dist-packages (from sklearn-contrib-py-earth==0.1.0+16.g400f84d) (1.4.1)

Requirement already satisfied: scikit-learn>=0.16 in /usr/local/lib/python3.7/dist-packages (from sklearn-contrib-py-earth==0.1.0+16.g400f84d) (0.22.2.post1)

Requirement already satisfied: six in /usr/local/lib/python3.7/dist-packages (from sklearn-contrib-py-earth==0.1.0+16.g400f84d) (1.15.0)

Requirement already satisfied: joblib>=0.11 in /usr/local/lib/python3.7/dist-packages (from scikit-learn>=0.16->sklearn-contrib-py-earth==0.1.0+16.g400f84d) (1.0.1)

Requirement already satisfied: numpy>=1.11.0 in /usr/local/lib/python3.7/dist-packages (from scikit-learn>=0.16->sklearn-contrib-py-earth==0.1.0+16.g400f84d) (1.19.5)

Building wheels for collected packages: sklearn-contrib-py-earth

Building wheel for sklearn-contrib-py-earth (setup.py) ... done

Created wheel for sklearn-contrib-py-earth: filename=sklearn_contrib_py_earth-0.1.0+16.g400f84d-cp37-cp37m-linux_x86_64.whl size=3997322 sha256=14

01d43c97053adbfd5f69f96fee6482c5704f5b329b6b7c17e11628004653b6

Stored in directory: /tmp/pip-ephem-wheel-cache-yzflnn8r/wheels/79/d7/a
a/1e418a9aea84424ef2ce2697ae7cccace322976388d3692e8e

Successfully built sklearn-contrib-py-earth

Installing collected packages: sklearn-contrib-py-earth

Successfully installed sklearn-contrib-py-earth-0.1.0+16.g400f84d

In [2]:

```
import numpy as np
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
import datetime
import warnings
import matplotlib.pyplot as plt
import statsmodels.api as sm
import math
import sklearn
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf as pltacf
from statsmodels.tsa.stattools import pacf as PACF
from statsmodels.graphics.tsaplots import plot_pacf as pltpacf
from pmdarima.arima import auto_arima
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error
warnings.filterwarnings('ignore')
```

In [3]:

```
!gdown --id 13L_plstV8yrYpjynopRunNVpageRgIMN
```

Downloading...

From: https://drive.google.com/uc?id=13L_plstV8yrYpjynopRunNVpageRgIMN

To: /content/SUNPHARMA.NS.csv

100% 157k/157k [00:00<00:00, 45.3MB/s]

In [4]:

```
stock_name="SUNPHARMA"
```

In [5]:

```
df_stock = pd.read_csv('SUNPHARMA.NS.csv', index_col='Date')
df_stock.head()
```

Out[5]:

	Open	High	Low	Close	Volume	Day of Week	Day of Month	Month	
Date									
2016-01-01	810.049988	817.900024	806.200012	815.549988	986728.0	4	1	1	779
2016-01-04	813.900024	821.299988	795.099976	799.099976	2388588.0	0	4	1	778
2016-01-05	807.000000	807.000000	792.049988	800.500000	1632394.0	1	5	1	774
2016-01-06	805.000000	805.799988	790.750000	791.950012	2168214.0	2	6	1	756
2016-01-07	788.000000	799.000000	778.650024	783.799988	2082334.0	3	7	1	760

ARIMA - Autoregression Intergrated Moving Average

Decomposing the data

In [6]:

```
df_stock['Close'].head()
```

Out[6]:

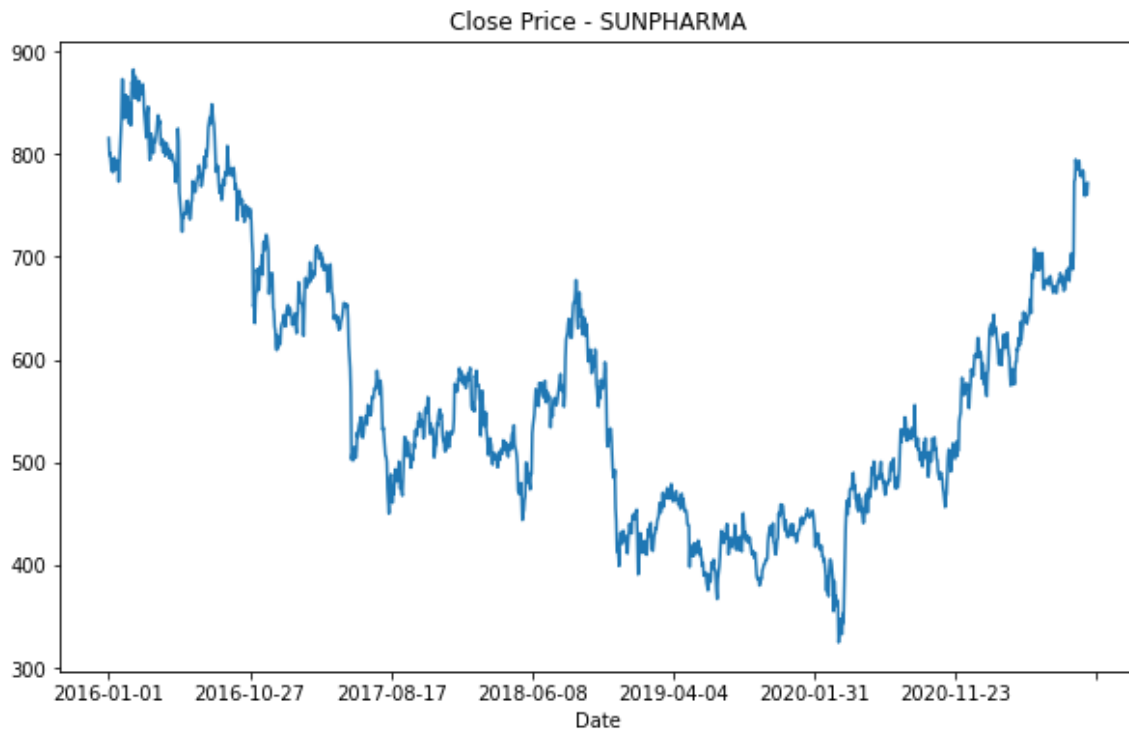
```
Date
2016-01-01    815.549988
2016-01-04    799.099976
2016-01-05    800.500000
2016-01-06    791.950012
2016-01-07    783.799988
Name: Close, dtype: float64
```

In [7]:

```
plt.rcParams['figure.figsize'] = (10,6)
df_stock['Close'].plot()
plt.title("Close Price - "+stock_name)
```

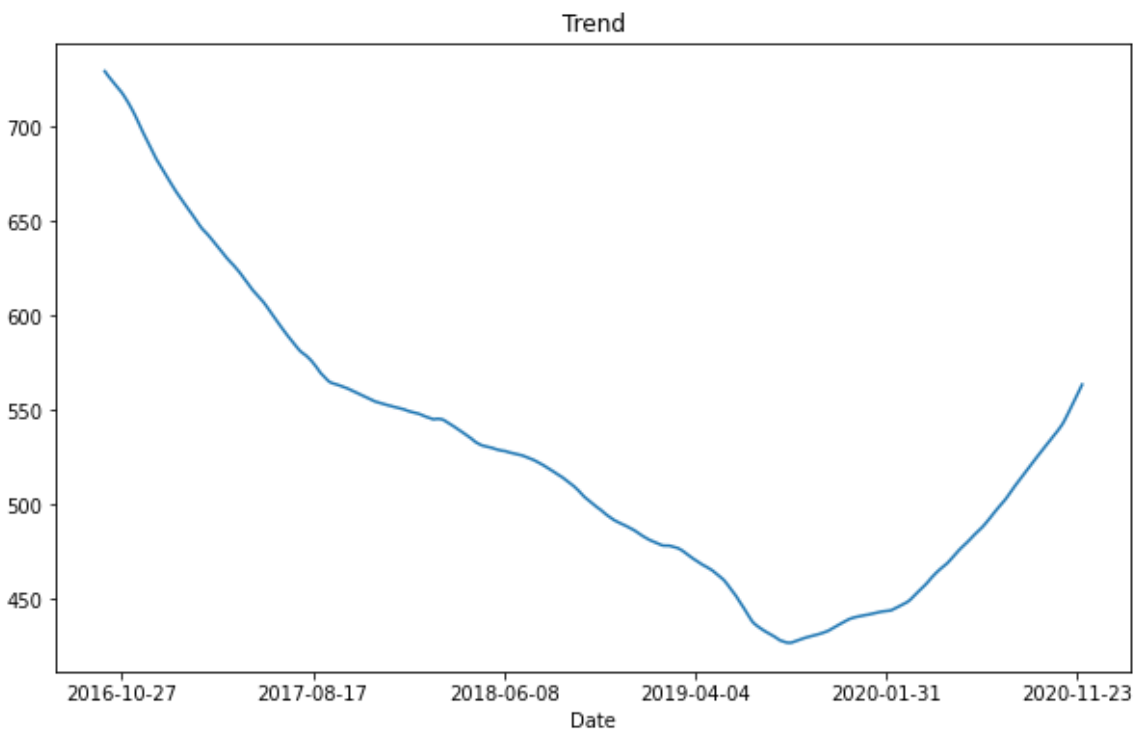
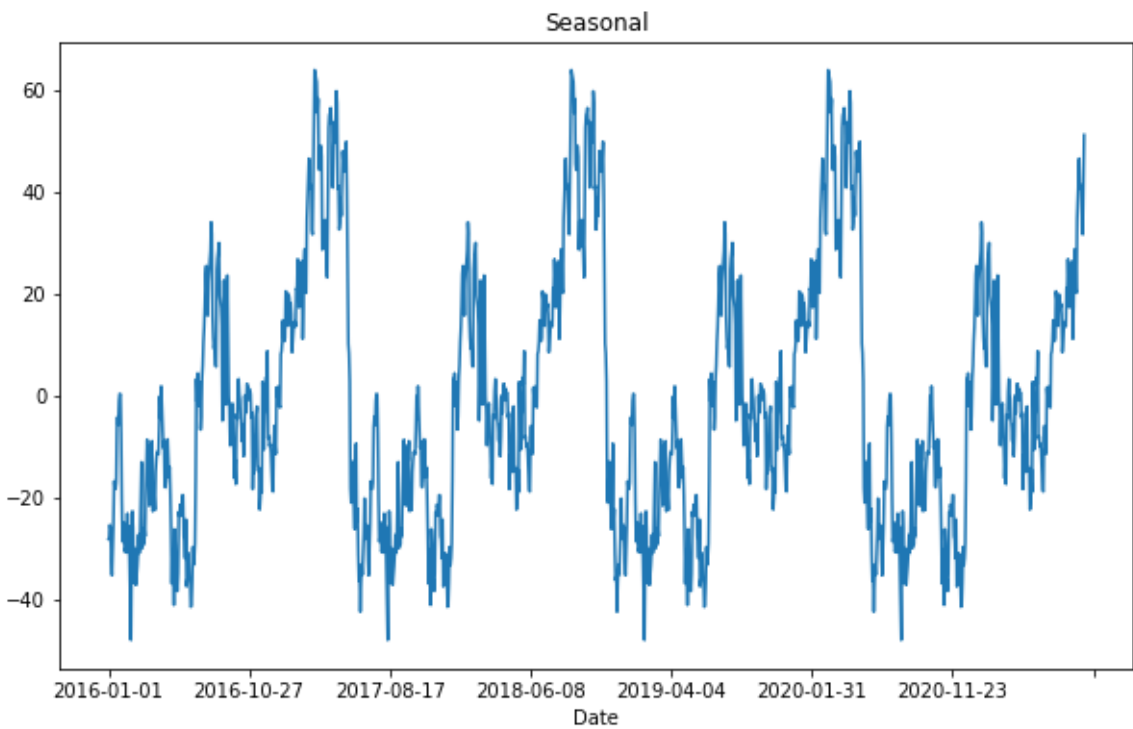
Out[7]:

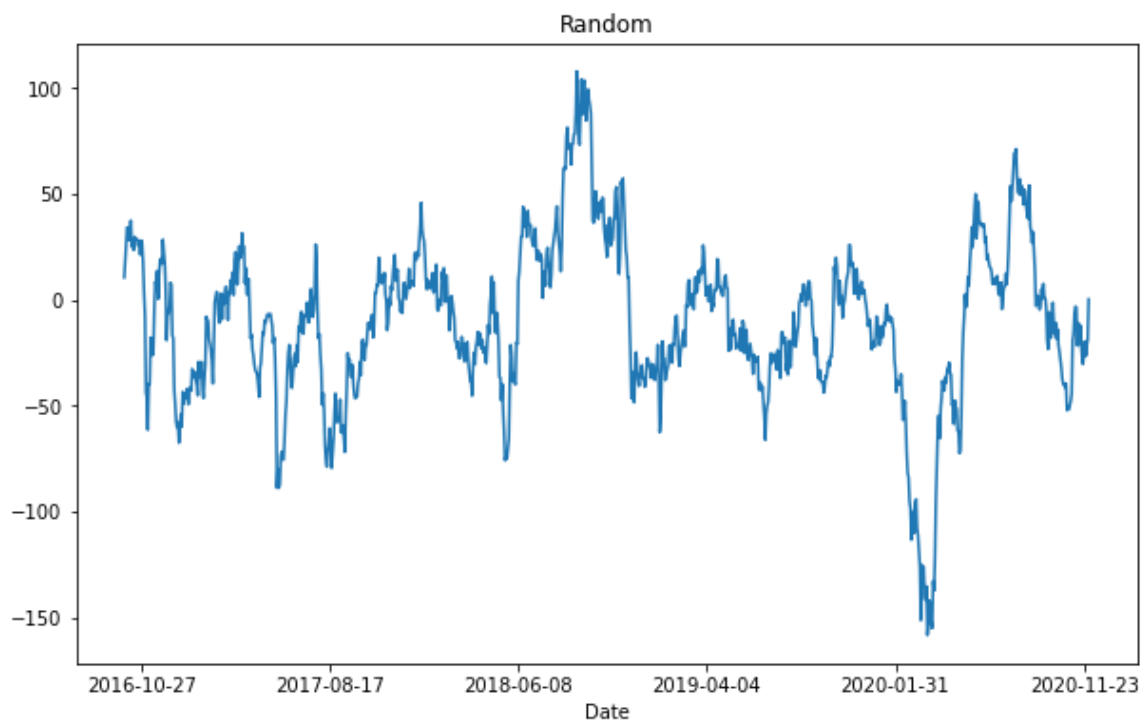
Text(0.5, 1.0, 'Close Price - SUNPHARMA')



In [8]:

```
plt.rcParams['figure.figsize'] = (10,6)
decomposition=seasonal_decompose(df_stock['Close'],freq=365)
decomposition.seasonal.plot()
plt.title("Seasonal")
plt.show()
decomposition.trend.plot()
plt.title("Trend")
plt.show()
decomposition.resid.plot()
plt.title("Random")
plt.show()
```





Stationary Check

In [9]:

```

Stationary=False
dvalue=0
Stationary_df=df_stock['Close'].copy()
while(Stationary==False):
    dfctest = adfuller(Stationary_df.dropna(),autolag='AIC')
    dfoutput=pd.Series(dfctest[0:4],index=['Test Statistic','p-value','#lags used','number
of observations used'])
    for key,value in dfctest[4].items():
        dfoutput['critical value (%s)'%key]= value
    print(dfctest[1])
    if (dfctest[1]>0.05):
        Stationary=False
        Stationary_df = Stationary_df - Stationary_df.shift(1)
    else :
        Stationary=True
    dvalue+=1
print(dfoutput)
print("Differencing Value : ",dvalue)

```

```

0.38774182764648624
Test Statistic          -1.785384
p-value                  0.387742
#lags used               0.000000
number of observations used 1387.000000
critical value (1%)      -3.435073
critical value (5%)      -2.863626
critical value (10%)     -2.567881
dtype: float64
Differencing Value : 1

```

ACF

In [10]:

```

from statsmodels.tsa.stattools import acf as ACF
ACFOutput = ACF(Stationary_df.dropna(),nlags=20)
print(ACFOutput)

```

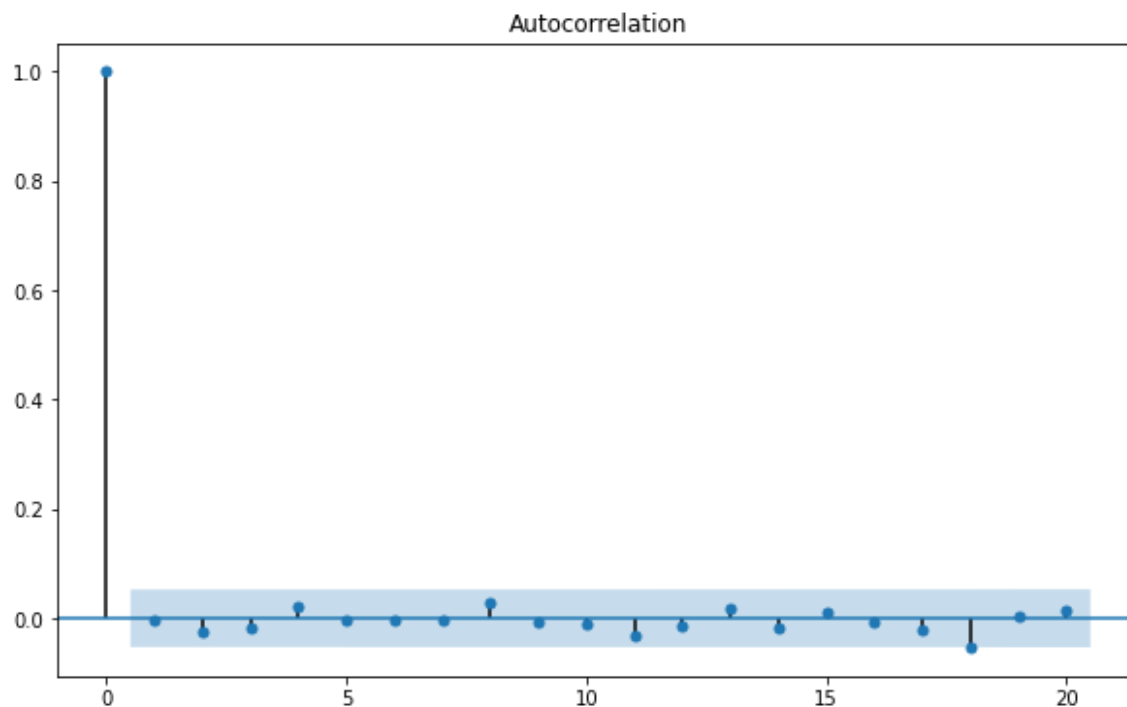
```

[ 1.          -0.0029988  -0.02534337 -0.01794135  0.01997992 -0.00273466
 -0.0031445  -0.00234666  0.02800393 -0.00771645 -0.01023255 -0.03222846
 -0.01439643  0.01761484 -0.01660349  0.0101602  -0.00695907 -0.02117723
 -0.05171695  0.00434775  0.01279746]

```

In [11]:

```
plt.rcParams['figure.figsize'] = (10,6)
plt.acf(Stationary_df.dropna(),lags=20)
plt.show()
```



PACF

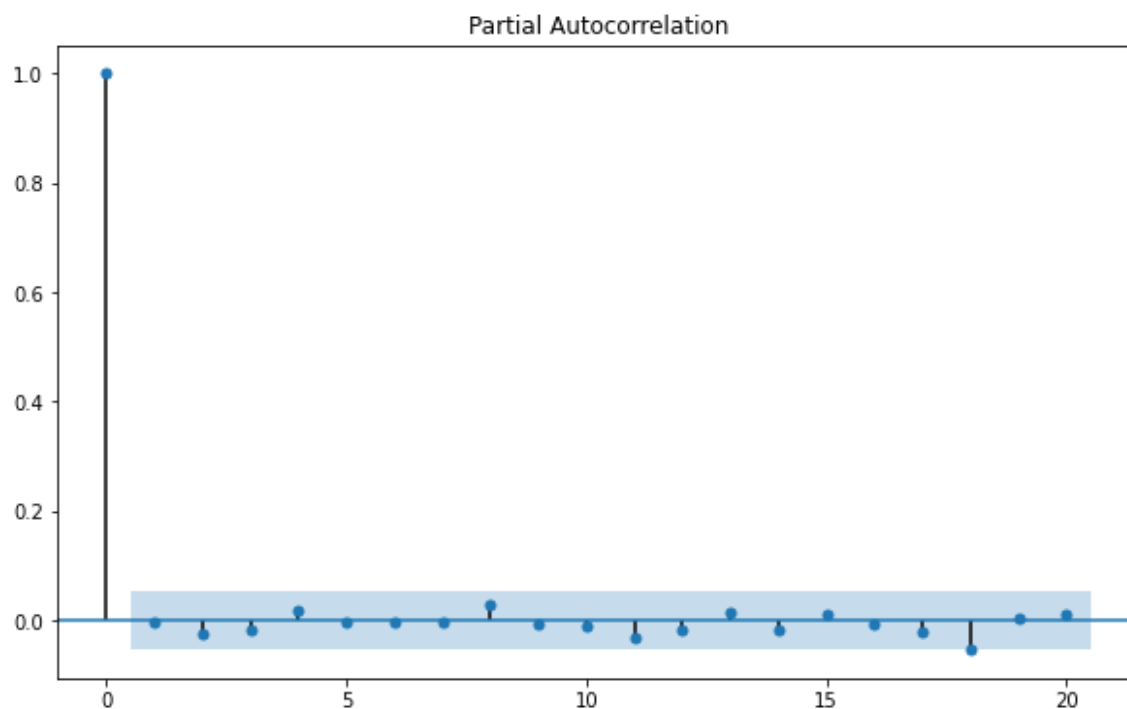
In [12]:

```
PACFOutput = PACF(Stationary_df.dropna(),nlags=20)
print(PACFOutput)
```

```
[ 1.          -0.00300096 -0.0253892  -0.01814639  0.0192928  -0.00353508
 -0.00251674 -0.00182647  0.02754971 -0.00769368 -0.00895849 -0.03193093
 -0.01662605  0.01623388 -0.01817111  0.01176077 -0.00770447 -0.02213255
 -0.05171037  0.00380047  0.01030138]
```

In [13]:

```
plt.rcParams['figure.figsize'] = (10,6)
plt.pacf(Stationary_df.dropna(),lags=20).show()
plt.show()
```



Test and Train Split

In [14]:

```
df_stock_train=df_stock.loc['2016-01-01':'2021-07-31']['Close']
df_stock_test=df_stock.loc['2021-07-31:']['Close']
```

In [15]:

```
df_stock_train.tail()
```

Out[15]:

```
Date
2021-07-26    703.400024
2021-07-27    688.299988
2021-07-28    687.799988
2021-07-29    703.000000
2021-07-30    773.950012
Name: Close, dtype: float64
```

In [16]:

```
df_stock_test.head()
```

Out[16]:

```
Date
2021-08-02    775.000000
2021-08-03    794.799988
2021-08-04    785.750000
2021-08-05    786.150024
2021-08-06    785.200012
Name: Close, dtype: float64
```

Model Fitting

In [17]:

```
arima_model=auto_arima(df_stock_train,seasonal=False,max_p=2,max_d=2,max_q=2,start_p=0,
start_q=0,d=1)
```

In [18]:

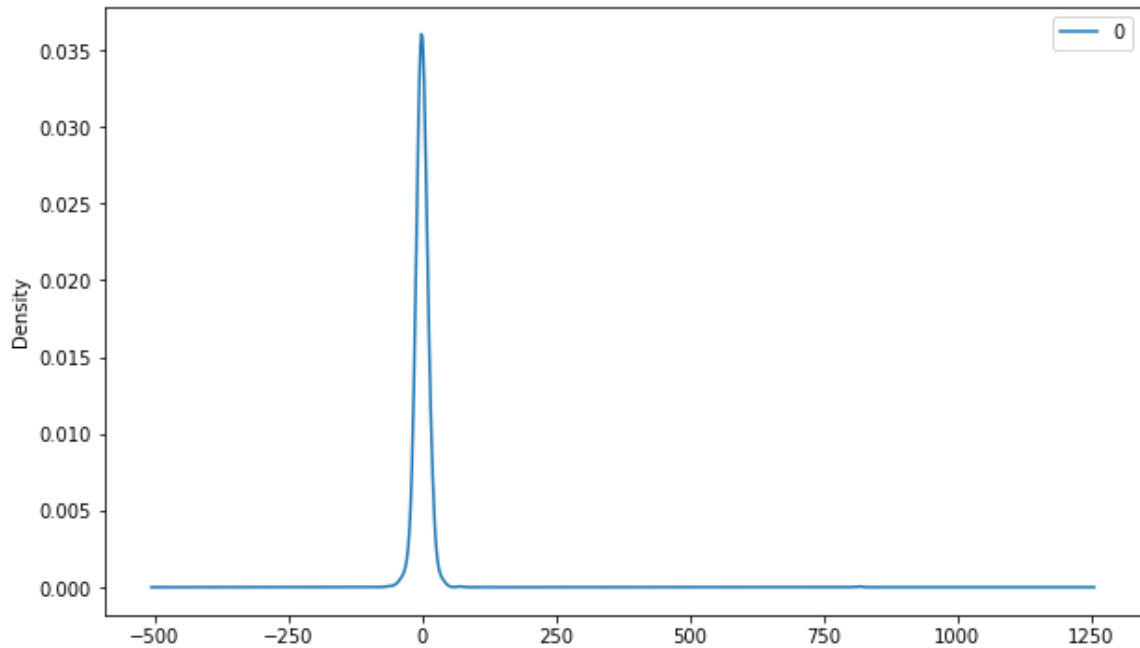
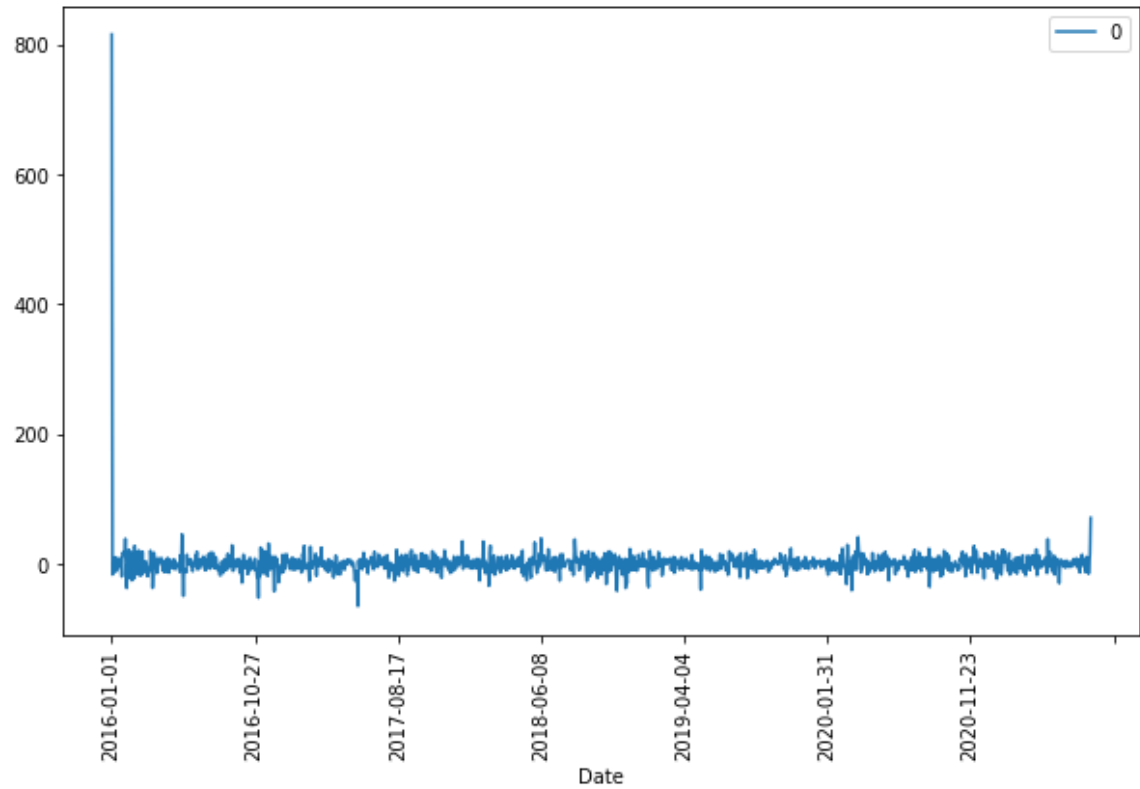
```
arima_model
```

Out[18]:

```
ARIMA(maxiter=50, method='lbfgs', order=(0, 1, 0), out_of_sample_size=0,
      scoring='mse', scoring_args={}, seasonal_order=(0, 0, 0, 0),
      start_params=None, suppress_warnings=True, trend=None,
      with_intercept=False)
```

In [19]:

```
model = ARIMA(df_stock_train, order=(0,1,0))
model_fit = model.fit()
residuals = pd.DataFrame(model_fit.resid)
plt.rcParams['figure.figsize'] = (10,6)
residuals.plot()
plt.xticks(rotation=90)
plt.show()
residuals.plot(kind='kde')
plt.show()
print(residuals.describe())
```



```
0
count    1369.000000
mean      0.565340
std       24.710264
min       -65.050018
25%       -6.299988
50%       -0.200012
75%        5.849976
max       815.549988
```

Forecasting and Testing

In [20]:

```
history = [x for x in df_stock_train.values]
predictions = list()
for t in range(len(df_stock_test.values)):
    model = ARIMA(history, order=(0,1,0))
    model_fit = model.fit()
    output = model_fit.forecast()
    yhat = output[0]
    predictions.append(yhat)
    obs = df_stock_test.values[t]
    history.append(obs)
error = np.sqrt(mean_squared_error(df_stock_test.values, predictions))
print('Test Error Percentage', (error/np.mean(df_stock_test.values))*100)
```

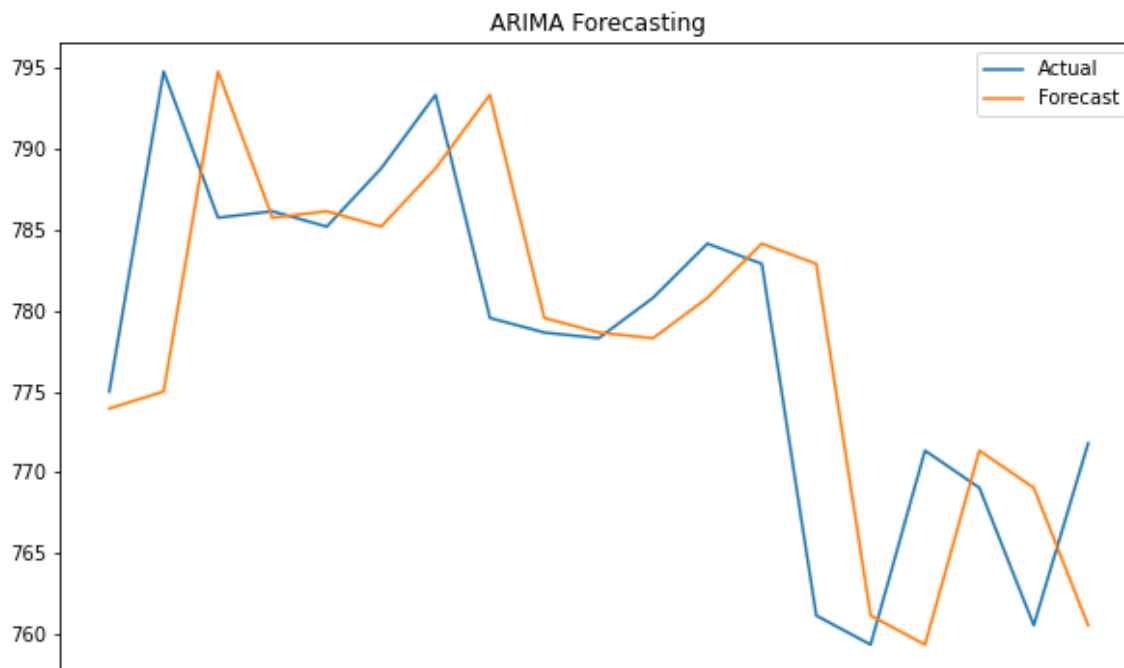
Test Error Percentage 1.1585437126034457

In [21]:

```
plt.rcParams['figure.figsize'] = (10,6)
plt.plot(df_stock_test, label='Actual')
plt.plot(predictions, label='Forecast')
plt.title('ARIMA Forecasting')
plt.legend()
plt.xticks([])
```

Out[21]:

([], <a list of 0 Text major ticklabel objects>)



Forecast for August

In [22]:

```
history = [x for x in df_stock_train.values]
predictions = list()
for t in range(len(df_stock_test.values[:14])):
    model = ARIMA(history, order=(1,1,1))
    model_fit = model.fit()
    output = model_fit.forecast()
    yhat = output[0]
    predictions.append(yhat)
    obs = df_stock_test.values[t]
    history.append(obs)
```

In [23]:

```
predictions
```

Out[23]:

```
[773.140853287741,
 774.2144455466643,
 794.1840073396351,
 784.9882749536607,
 785.4131809065703,
 784.4912634600369,
 788.1045294044699,
 792.6673739998397,
 778.9286182349283,
 778.0533364729878,
 777.7232790835747,
 780.2307903412211,
 783.583850426303,
 782.3571507599959]
```

MARS - Multivariate Adaptive Regression Splines

In [24]:

```
from pyearth import Earth
```

In [25]:

```
df_stock_mars_train=df_stock.loc['2016-01-01':'2021-07-31'].iloc[:,5].copy()
df_stock_mars_test=df_stock.loc['2021-07-31:'].iloc[:,5].copy()
```

In [26]:

```
df_stock_mars_train.shape
```

Out[26]:

```
(1369, 5)
```

In [27]:

```
df_stock_mars_test.shape
```

Out[27]:

(19, 5)

In [28]:

```
df_stock_mars_train.head()
```

Out[28]:

	Open	High	Low	Close	Volume
Date					
2016-01-01	810.049988	817.900024	806.200012	815.549988	986728.0
2016-01-04	813.900024	821.299988	795.099976	799.099976	2388588.0
2016-01-05	807.000000	807.000000	792.049988	800.500000	1632394.0
2016-01-06	805.000000	805.799988	790.750000	791.950012	2168214.0
2016-01-07	788.000000	799.000000	778.650024	783.799988	2082334.0

In [29]:

```
df_stock_mars_test
```

Out[29]:

	Open	High	Low	Close	Volume
Date					
2021-08-02	785.000000	801.900024	764.099976	775.000000	18661462.0
2021-08-03	773.950012	804.349976	765.250000	794.799988	14678219.0
2021-08-04	791.000000	797.900024	779.700012	785.750000	5796169.0
2021-08-05	785.500000	794.400024	783.299988	786.150024	5026619.0
2021-08-06	786.500000	796.450012	781.549988	785.200012	4064324.0
2021-08-09	789.150024	793.000000	782.000000	788.799988	2873586.0
2021-08-10	789.950012	798.799988	782.200012	793.349976	5656020.0
2021-08-11	794.000000	795.000000	773.299988	779.549988	4275776.0
2021-08-12	780.000000	781.250000	773.150024	778.650024	2821380.0
2021-08-13	769.000000	788.500000	767.650024	778.299988	4319920.0
2021-08-16	775.200012	783.000000	771.049988	780.799988	2310304.0
2021-08-17	783.000000	793.200012	778.200012	784.150024	5686055.0
2021-08-18	788.000000	791.400024	779.299988	782.900024	4036305.0
2021-08-20	775.000000	784.450012	743.349976	761.150024	6238261.0
2021-08-23	765.000000	770.000000	750.799988	759.349976	2375417.0
2021-08-24	759.799988	777.950012	754.250000	771.349976	2732537.0
2021-08-25	770.000000	776.000000	759.599976	769.049988	2254852.0
2021-08-26	765.150024	771.099976	759.250000	760.549988	1735864.0
2021-08-27	760.750000	777.000000	758.099976	771.799988	2432862.0

In [30]:

```
x_train_mars = df_stock_mars_train.drop(['Close'], axis=1)
y_train_mars = df_stock_mars_train['Close']
x_test_mars = df_stock_mars_test.drop(['Close'], axis=1)
y_test_mars = df_stock_mars_test['Close']
```

Fitting the model

In [31]:

```
MARS_model = Earth(max_terms = 600,max_degree=3)
MARS_model_fitted = MARS_model.fit(x_train_mars, y_train_mars)
y_pred_mars = MARS_model_fitted.predict(x_test_mars[:14])
```

Evaluating the model

In [32]:

```
MSE = mean_squared_error(y_test_mars[:14], y_pred_mars)
RMSE = np.sqrt(MSE)
RMSE_mean = (RMSE/np.mean(y_test_mars[:14]))*100
print("RMSE Score :", round(RMSE, 2))
print("RMSE/Mean", round(RMSE_mean,1))
```

RMSE Score : 4.33

RMSE/Mean 0.6

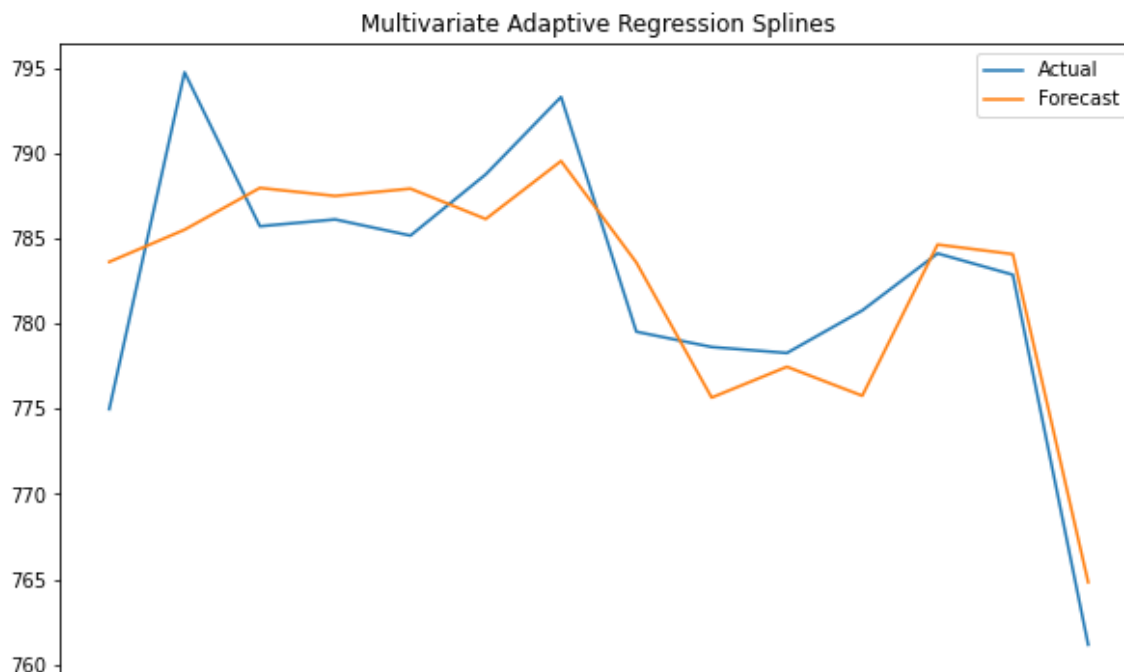
Plotting Forecast vs Actual

In [33]:

```
plt.rcParams['figure.figsize'] = (10,6)
plt.plot(y_test_mars[:14], label='Actual')
plt.plot(y_pred_mars, label='Forecast')
plt.title('Multivariate Adaptive Regression Splines')
plt.legend()
plt.xticks([])
```

Out[33]:

([], <a list of 0 Text major ticklabel objects>)



VAR - Vector Autoregression

In [34]:

```
from statsmodels.tsa.api import VAR
from statsmodels.tsa.stattools import adfuller
from statsmodels.tools.eval_measures import rmse, aic
```

In [35]:

```
df_stock_var = df_stock.loc['2016-01-01':'2021-08-20'].iloc[:,4].copy()
df_stock_var_train=df_stock.loc['2016-01-01':'2021-07-31'].iloc[:,4].copy()
df_stock_var_test=df_stock.loc['2021-07-31':'2021-08-20'].iloc[:,4].copy()
```

In [36]:

```
df_stock_var_train.shape
```

Out[36]:

```
(1369, 4)
```

In [37]:

```
df_stock_var_test.shape
```

Out[37]:

```
(14, 4)
```

In [38]:

```
df_stock_var.shape
```

Out[38]:

```
(1383, 4)
```

Plotting the Data

In [39]:

```
# fig, axes = plt.subplots(nrows=4, ncols=1, dpi=120, figsize=(10,6))
# for i, ax in enumerate(axes.flatten()):
#     data = df_stock_var[df_stock_var.columns[i]]
#     ax.plot(data, linewidth=1)
#     # Decorations
#     ax.set_title(df_stock_var.columns[i])
#     ax.xaxis.set_ticks_position('none')
#     ax.yaxis.set_ticks_position('none')
#     ax.spines["top"].set_alpha(0)
#     ax.tick_params(labelsize=6)

# plt.tight_layout()
```

Granger Causality Test

In [40]:

```

from statsmodels.tsa.stattools import grangercausalitytests
maxlag=12
test = 'ssr_chi2test'
def grangers_causation_matrix(data, variables, test='ssr_chi2test', verbose=False):
    df = pd.DataFrame(np.zeros((len(variables), len(variables))), columns=variables, index=variables)
    for c in df.columns:
        for r in df.index:
            test_result = grangercausalitytests(data[[r, c]], maxlag=maxlag, verbose=False)
            p_values = [round(test_result[i+1][0][test][1],4) for i in range(maxlag)]
            if verbose: print(f'Y = {r}, X = {c}, P Values = {p_values}')
            min_p_value = np.min(p_values)
            df.loc[r, c] = min_p_value
    df.columns = [var + '_x' for var in variables]
    df.index = [var + '_y' for var in variables]
    return df

grangers_causation_matrix(df_stock_var, variables = df_stock_var.columns)

```

Out[40]:

	Open_x	High_x	Low_x	Close_x
Open_y	1.0000	0.0000	0.000	0.0
High_y	0.0000	1.0000	0.000	0.0
Low_y	0.0000	0.0000	1.000	0.0
Close_y	0.0312	0.0011	0.315	1.0

Cointegration Test

In [41]:

```

from statsmodels.tsa.vector_ar.vecm import coint_johansen

def cointegration_test(df_stock_var, alpha=0.05):
    out = coint_johansen(df_stock_var, -1, 5)
    d = {'0.90':0, '0.95':1, '0.99':2}
    traces = out.lr1
    cvts = out.cvt[:, d[str(1-alpha)]]
    def adjust(val, length= 6): return str(val).ljust(length)

    print('Name    :: Test Statistic > C(95%)    => Significance \n', '--'*20)
    for col, trace, cvt in zip(df_stock_var.columns, traces, cvts):
        print(adjust(col), ':: ', adjust(round(trace,2), 9), ">", adjust(cvt, 8), ' =>
', trace > cvt)

cointegration_test(df_stock_var)

```

Name	::	Test Statistic > C(95%)	=>	Significance
Open	::	502.29 > 40.1749	=>	True
High	::	258.06 > 24.2761	=>	True
Low	::	86.79 > 12.3212	=>	True
Close	::	0.2 > 4.1296	=>	False

ADF Test

In [42]:

```

def adfuller_test(series, signif=0.05, name='', verbose=False):
    """Perform ADFuller to test for Stationarity of given series and print report"""
    r = adfuller(series, autolag='AIC')
    output = {'test_statistic':round(r[0], 4), 'pvalue':round(r[1], 4), 'n_lags':round(
r[2], 4), 'n_obs':r[3]}
    p_value = output['pvalue']
    def adjust(val, length= 6): return str(val).ljust(length)

    # Print Summary
    print(f'    Augmented Dickey-Fuller Test on "{name}"', "\n    ", '-'*47)
    print(f' Null Hypothesis: Data has unit root. Non-Stationary.')
    print(f' Significance Level    = {signif}')
    print(f' Test Statistic          = {output["test_statistic"]}')
    print(f' No. Lags Chosen          = {output["n_lags"]}')

    for key,val in r[4].items():
        print(f' Critical value {adjust(key)} = {round(val, 3)}')

    if p_value <= signif:
        print(f" => P-Value = {p_value}. Rejecting Null Hypothesis.")
        print(f" => Series is Stationary.")
    else:
        print(f" => P-Value = {p_value}. Weak evidence to reject the Null Hypothesis.")
        print(f" => Series is Non-Stationary.")

```

In [43]:

```
# ADF Test on each column
for name, column in df_stock_var_train.iteritems():
    adfuller_test(column, name=column.name)
    print('\n')
```

Augmented Dickey-Fuller Test on "Open"

Null Hypothesis: Data has unit root. Non-Stationary.

Significance Level = 0.05

Test Statistic = -1.9827

No. Lags Chosen = 1

Critical value 1% = -3.435

Critical value 5% = -2.864

Critical value 10% = -2.568

=> P-Value = 0.2942. Weak evidence to reject the Null Hypothesis.

=> Series is Non-Stationary.

Augmented Dickey-Fuller Test on "High"

Null Hypothesis: Data has unit root. Non-Stationary.

Significance Level = 0.05

Test Statistic = -1.6054

No. Lags Chosen = 2

Critical value 1% = -3.435

Critical value 5% = -2.864

Critical value 10% = -2.568

=> P-Value = 0.4809. Weak evidence to reject the Null Hypothesis.

=> Series is Non-Stationary.

Augmented Dickey-Fuller Test on "Low"

Null Hypothesis: Data has unit root. Non-Stationary.

Significance Level = 0.05

Test Statistic = -1.9492

No. Lags Chosen = 3

Critical value 1% = -3.435

Critical value 5% = -2.864

Critical value 10% = -2.568

=> P-Value = 0.3093. Weak evidence to reject the Null Hypothesis.

=> Series is Non-Stationary.

Augmented Dickey-Fuller Test on "Close"

Null Hypothesis: Data has unit root. Non-Stationary.

Significance Level = 0.05

Test Statistic = -1.793

No. Lags Chosen = 0

Critical value 1% = -3.435

Critical value 5% = -2.864

Critical value 10% = -2.568

=> P-Value = 0.384. Weak evidence to reject the Null Hypothesis.

=> Series is Non-Stationary.

ADF Test on first order differencing

In [44]:

```
df_differenced = df_stock_var_train.diff().dropna()
```

In [45]:

```
# ADF Test on each column of 1st Differences Dataframe
for name, column in df_differenced.iteritems():
    adfuller_test(column, name=column.name)
    print('\n')
```

Augmented Dickey-Fuller Test on "Open"

Null Hypothesis: Data has unit root. Non-Stationary.
 Significance Level = 0.05
 Test Statistic = -40.5175
 No. Lags Chosen = 0
 Critical value 1% = -3.435
 Critical value 5% = -2.864
 Critical value 10% = -2.568
 => P-Value = 0.0. Rejecting Null Hypothesis.
 => Series is Stationary.

Augmented Dickey-Fuller Test on "High"

Null Hypothesis: Data has unit root. Non-Stationary.
 Significance Level = 0.05
 Test Statistic = -25.3361
 No. Lags Chosen = 1
 Critical value 1% = -3.435
 Critical value 5% = -2.864
 Critical value 10% = -2.568
 => P-Value = 0.0. Rejecting Null Hypothesis.
 => Series is Stationary.

Augmented Dickey-Fuller Test on "Low"

Null Hypothesis: Data has unit root. Non-Stationary.
 Significance Level = 0.05
 Test Statistic = -21.3225
 No. Lags Chosen = 2
 Critical value 1% = -3.435
 Critical value 5% = -2.864
 Critical value 10% = -2.568
 => P-Value = 0.0. Rejecting Null Hypothesis.
 => Series is Stationary.

Augmented Dickey-Fuller Test on "Close"

Null Hypothesis: Data has unit root. Non-Stationary.
 Significance Level = 0.05
 Test Statistic = -36.4988
 No. Lags Chosen = 0
 Critical value 1% = -3.435
 Critical value 5% = -2.864
 Critical value 10% = -2.568
 => P-Value = 0.0. Rejecting Null Hypothesis.
 => Series is Stationary.

Selecting the number of laas

In [46]:

```
model = VAR(df_differenced)
x = model.select_order(maxlags=20)
x.summary()
```

Out[46]:

VAR Order Selection (* highlights the minimums)

	AIC	BIC	FPE	HQIC
0	16.78	16.79	1.936e+07	16.78
1	14.96	15.04	3.150e+06	14.99
2	14.51	14.64	1.994e+06	14.56
3	14.25	14.45	1.545e+06	14.33
4	14.18	14.44	1.438e+06	14.28
5	14.08	14.41*	1.307e+06	14.20
6	14.03	14.42	1.240e+06	14.18
7	13.99	14.44	1.189e+06	14.16*
8	13.98	14.49	1.184e+06	14.18
9	13.98	14.55	1.176e+06	14.19
10	13.97	14.60	1.166e+06	14.21
11	13.96	14.66	1.160e+06	14.22
12	13.97	14.72	1.163e+06	14.25
13	13.97	14.79	1.172e+06	14.28
14	13.96*	14.84	1.153e+06*	14.29
15	13.97	14.91	1.169e+06	14.32
16	13.97	14.97	1.168e+06	14.35
17	13.97	15.04	1.169e+06	14.37
18	13.97	15.10	1.173e+06	14.40
19	13.98	15.17	1.185e+06	14.43
20	13.99	15.24	1.193e+06	14.46

Fitting the model

In [81]:

```
model_fitted = model.fit(14)
model_fitted.summary()
```

Out[81]:

Summary of Regression Results

```
=====
Model:                VAR
Method:               OLS
Date:                Tue, 14, Sep, 2021
Time:                14:48:02
```

```
-----
No. of Equations:    4.00000    BIC:                14.8429
Nobs:                1354.00    HQIC:              14.2940
Log likelihood:      -16911.6    FPE:               1.16198e+06
AIC:                 13.9654    Det(Omega_mle):    985295.
```

Results for equation Open

```
=====
==
```

	coefficient	std. error	t-stat	pr
ob				
--				
const	-0.018363	0.134011	-0.137	0.8
91				
L1.Open	-0.983858	0.038791	-25.363	0.0
00				
L1.High	0.061084	0.037496	1.629	0.1
03				
L1.Low	0.007992	0.030357	0.263	0.7
92				
L1.Close	1.005162	0.034121	29.459	0.0
00				
L2.Open	-0.934407	0.053783	-17.374	0.0
00				
L2.High	0.119258	0.049912	2.389	0.0
17				
L2.Low	0.003987	0.041738	0.096	0.9
24				
L2.Close	0.866511	0.055038	15.744	0.0
00				
L3.Open	-0.790179	0.063942	-12.358	0.0
00				
L3.High	0.170696	0.058050	2.940	0.0
03				
L3.Low	-0.024545	0.049656	-0.494	0.6
21				
L3.Close	0.809471	0.067940	11.914	0.0
00				
L4.Open	-0.789100	0.069579	-11.341	0.0
00				
L4.High	0.148492	0.063586	2.335	0.0
20				
L4.Low	0.029849	0.054556	0.547	0.5
84				
L4.Close	0.613632	0.077228	7.946	0.0
00				
L5.Open	-0.642814	0.073427	-8.754	0.0
00				
L5.High	0.105774	0.066976	1.579	0.1
14				
L5.Low	-0.013916	0.057452	-0.242	0.8
09				

L5.Close 00	0.652748	0.082962	7.868	0.0
L6.Open 00	-0.544984	0.075481	-7.220	0.0
L6.High 64	0.050595	0.069037	0.733	0.4
L6.Low 24	-0.005648	0.059485	-0.095	0.9
L6.Close 00	0.551687	0.087730	6.288	0.0
L7.Open 00	-0.456741	0.076241	-5.991	0.0
L7.High 46	0.065832	0.069927	0.941	0.3
L7.Low 03	-0.015055	0.060462	-0.249	0.8
L7.Close 00	0.481791	0.090545	5.321	0.0
L8.Open 00	-0.439371	0.076324	-5.757	0.0
L8.High 50	0.135540	0.069286	1.956	0.0
L8.Low 59	0.044758	0.060409	0.741	0.4
L8.Close 01	0.312349	0.090806	3.440	0.0
L9.Open 00	-0.376620	0.075439	-4.992	0.0
L9.High 66	0.123707	0.067369	1.836	0.0
L9.Low 47	0.019276	0.059649	0.323	0.7
L9.Close 03	0.259860	0.088297	2.943	0.0
L10.Open 00	-0.276385	0.072706	-3.801	0.0
L10.High 16	0.101894	0.064875	1.571	0.1
L10.Low 56	0.017895	0.057481	0.311	0.7
L10.Close 07	0.227761	0.084351	2.700	0.0
L11.Open 00	-0.243499	0.068391	-3.560	0.0
L11.High 72	0.067364	0.061374	1.098	0.2
L11.Low 77	-0.015484	0.054558	-0.284	0.7
L11.Close 25	0.175112	0.078387	2.234	0.0
L12.Open 09	-0.159749	0.060898	-2.623	0.0
L12.High 18	0.087628	0.056127	1.561	0.1
L12.Low 69	-0.001965	0.049949	-0.039	0.9
L12.Close 20	0.165154	0.070824	2.332	0.0
L13.Open 06	-0.136239	0.049205	-2.769	0.0
L13.High	0.061977	0.048277	1.284	0.1

```

99
L13.Low      0.048087      0.042174      1.140      0.2
54
L13.Close    0.060270      0.059315      1.016      0.3
10
L14.Open     -0.060833      0.029625     -2.053      0.0
40
L14.High     0.032203      0.036649      0.879      0.3
80
L14.Low      0.024341      0.030583      0.796      0.4
26
L14.Close    0.036445      0.043736      0.833      0.4
05

```

```

=====
==

```

Results for equation High

```

=====

```

```

==

```

	coefficient	std. error	t-stat	pr
ob				

--				
const	-0.007202	0.216194	-0.033	0.9
73				
L1.Open	-0.004068	0.062579	-0.065	0.9
48				
L1.High	-0.866007	0.060491	-14.316	0.0
00				
L1.Low	-0.142393	0.048973	-2.908	0.0
04				
L1.Close	1.024308	0.055046	18.608	0.0
00				
L2.Open	0.111600	0.086765	1.286	0.1
98				
L2.High	-0.802965	0.080521	-9.972	0.0
00				
L2.Low	-0.243692	0.067334	-3.619	0.0
00				
L2.Close	1.017245	0.088791	11.457	0.0
00				
L3.Open	0.038891	0.103155	0.377	0.7
06				
L3.High	-0.617304	0.093650	-6.592	0.0
00				
L3.Low	-0.312473	0.080108	-3.901	0.0
00				
L3.Close	0.879593	0.109606	8.025	0.0
00				
L4.Open	-0.086121	0.112249	-0.767	0.4
43				
L4.High	-0.611963	0.102581	-5.966	0.0
00				
L4.Low	-0.255223	0.088014	-2.900	0.0
04				
L4.Close	0.869557	0.124588	6.979	0.0
00				
L5.Open	0.030111	0.118457	0.254	0.7
99				
L5.High	-0.582007	0.108050	-5.386	0.0
00				

L5.Low 02	-0.290378	0.092685	-3.133	0.0
L5.Close 00	0.924238	0.133839	6.906	0.0
L6.Open 74	0.068452	0.121770	0.562	0.5
L6.High 00	-0.616584	0.111374	-5.536	0.0
L6.Low 01	-0.327117	0.095965	-3.409	0.0
L6.Close 00	0.901449	0.141531	6.369	0.0
L7.Open 06	0.081870	0.122997	0.666	0.5
L7.High 00	-0.442676	0.112810	-3.924	0.0
L7.Low 02	-0.297921	0.097540	-3.054	0.0
L7.Close 00	0.756098	0.146072	5.176	0.0
L8.Open 35	0.058533	0.123130	0.475	0.6
L8.High 01	-0.380356	0.111776	-3.403	0.0
L8.Low 19	-0.229240	0.097456	-2.352	0.0
L8.Close 00	0.579047	0.146493	3.953	0.0
L9.Open 46	0.039473	0.121702	0.324	0.7
L9.High 04	-0.313578	0.108683	-2.885	0.0
L9.Low 70	-0.174234	0.096230	-1.811	0.0
L9.Close 01	0.478098	0.142446	3.356	0.0
L10.Open 93	0.046338	0.117294	0.395	0.6
L10.High 78	-0.184367	0.104660	-1.762	0.0
L10.Low 72	-0.126718	0.092732	-1.367	0.1
L10.Close 15	0.331174	0.136080	2.434	0.0
L11.Open 18	-0.025376	0.110333	-0.230	0.8
L11.High 13	-0.246953	0.099012	-2.494	0.0
L11.Low 94	-0.147616	0.088016	-1.677	0.0
L11.Close 14	0.311773	0.126458	2.465	0.0
L12.Open 92	0.001000	0.098245	0.010	0.9
L12.High 34	-0.135655	0.090547	-1.498	0.1
L12.Low 94	-0.134800	0.080581	-1.673	0.0
L12.Close 03	0.344365	0.114257	3.014	0.0
L13.Open	-0.045089	0.079380	-0.568	0.5

70				
L13.High	-0.088302	0.077883	-1.134	0.2
57				
L13.Low	0.014934	0.068038	0.219	0.8
26				
L13.Close	0.170044	0.095691	1.777	0.0
76				
L14.Open	-0.034434	0.047793	-0.720	0.4
71				
L14.High	-0.026928	0.059125	-0.455	0.6
49				
L14.Low	0.009630	0.049338	0.195	0.8
45				
L14.Close	0.079706	0.070557	1.130	0.2
59				

=====

==

Results for equation Low

=====

==

	coefficient	std. error	t-stat	pr
ob				
--				
const	-0.037569	0.244621	-0.154	0.8
78				
L1.Open	0.010660	0.070808	0.151	0.8
80				
L1.High	-0.202733	0.068445	-2.962	0.0
03				
L1.Low	-0.906544	0.055412	-16.360	0.0
00				
L1.Close	1.133956	0.062284	18.206	0.0
00				
L2.Open	-0.004730	0.098174	-0.048	0.9
62				
L2.High	-0.114946	0.091109	-1.262	0.2
07				
L2.Low	-0.790624	0.076188	-10.377	0.0
00				
L2.Close	0.949802	0.100466	9.454	0.0
00				
L3.Open	0.025023	0.116719	0.214	0.8
30				
L3.High	-0.007740	0.105964	-0.073	0.9
42				
L3.Low	-0.682337	0.090641	-7.528	0.0
00				
L3.Close	0.805659	0.124018	6.496	0.0
00				
L4.Open	-0.128818	0.127008	-1.014	0.3
10				
L4.High	-0.014250	0.116069	-0.123	0.9
02				
L4.Low	-0.543945	0.099587	-5.462	0.0
00				
L4.Close	0.630298	0.140970	4.471	0.0
00				
L5.Open	-0.031620	0.134033	-0.236	0.8
14				

L5.High 23	-0.043394	0.122258	-0.355	0.7
L5.Low 00	-0.554956	0.104873	-5.292	0.0
L5.Close 00	0.680391	0.151438	4.493	0.0
L6.Open 41	-0.106228	0.137782	-0.771	0.4
L6.High 37	-0.097854	0.126019	-0.777	0.4
L6.Low 00	-0.416680	0.108583	-3.837	0.0
L6.Close 00	0.565943	0.160141	3.534	0.0
L7.Open 66	-0.005859	0.139170	-0.042	0.9
L7.High 98	-0.049614	0.127644	-0.389	0.6
L7.Low 00	-0.449111	0.110366	-4.069	0.0
L7.Close 00	0.607233	0.165279	3.674	0.0
L8.Open 16	-0.032389	0.139321	-0.232	0.8
L8.High 08	0.014541	0.126474	0.115	0.9
L8.Low 12	-0.276828	0.110270	-2.510	0.0
L8.Close 25	0.370474	0.165756	2.235	0.0
L9.Open 19	-0.068399	0.137705	-0.497	0.6
L9.High 43	0.024412	0.122974	0.199	0.8
L9.Low 06	-0.300344	0.108883	-2.758	0.0
L9.Close 69	0.293133	0.161177	1.819	0.0
L10.Open 35	0.027719	0.132717	0.209	0.8
L10.High 04	0.061377	0.118422	0.518	0.6
L10.Low 44	-0.210865	0.104925	-2.010	0.0
L10.Close 98	0.254941	0.153974	1.656	0.0
L11.Open 22	-0.100207	0.124840	-0.803	0.4
L11.High 90	0.077410	0.112031	0.691	0.4
L11.Low 90	-0.130383	0.099590	-1.309	0.1
L11.Close 34	0.029909	0.143086	0.209	0.8
L12.Open 89	-0.095822	0.111163	-0.862	0.3
L12.High 58	0.193931	0.102453	1.893	0.0
L12.Low 54	-0.084434	0.091177	-0.926	0.3
L12.Close	0.061731	0.129281	0.477	0.6

33				
L13.Open	-0.160399	0.089818	-1.786	0.0
74				
L13.High	0.167214	0.088124	1.897	0.0
58				
L13.Low	0.060705	0.076984	0.789	0.4
30				
L13.Close	-0.082941	0.108273	-0.766	0.4
44				
L14.Open	-0.065934	0.054077	-1.219	0.2
23				
L14.High	0.131035	0.066899	1.959	0.0
50				
L14.Low	-0.016097	0.055825	-0.288	0.7
73				
L14.Close	-0.028873	0.079834	-0.362	0.7
18				

=====

==

Results for equation Close

=====

==

	coefficient	std. error	t-stat	pr
ob				
--				
const	-0.007891	0.302901	-0.026	0.9
79				
L1.Open	0.047169	0.087677	0.538	0.5
91				
L1.High	-0.129018	0.084752	-1.522	0.1
28				
L1.Low	-0.070883	0.068614	-1.033	0.3
02				
L1.Close	0.132694	0.077123	1.721	0.0
85				
L2.Open	0.184635	0.121564	1.519	0.1
29				
L2.High	-0.137779	0.112815	-1.221	0.2
22				
L2.Low	-0.087337	0.094339	-0.926	0.3
55				
L2.Close	0.118609	0.124402	0.953	0.3
40				
L3.Open	0.186237	0.144527	1.289	0.1
98				
L3.High	0.019567	0.131209	0.149	0.8
81				
L3.Low	-0.153881	0.112236	-1.371	0.1
70				
L3.Close	-0.031207	0.153564	-0.203	0.8
39				
L4.Open	0.051741	0.157268	0.329	0.7
42				
L4.High	-0.093859	0.143722	-0.653	0.5
14				
L4.Low	-0.097415	0.123313	-0.790	0.4
30				
L4.Close	0.032278	0.174556	0.185	0.8
53				

L5.Open 56	0.188340	0.165966	1.135	0.2
L5.High 51	-0.114209	0.151385	-0.754	0.4
L5.Low 45	-0.150977	0.129858	-1.163	0.2
L5.Close 29	0.148151	0.187517	0.790	0.4
L6.Open 84	0.148416	0.170608	0.870	0.3
L6.High 24	-0.240288	0.156042	-1.540	0.1
L6.Low 39	-0.158295	0.134453	-1.177	0.2
L6.Close 06	0.164641	0.198294	0.830	0.4
L7.Open 28	0.207639	0.172326	1.205	0.2
L7.High 20	-0.127418	0.158055	-0.806	0.4
L7.Low 42	-0.200456	0.136660	-1.467	0.1
L7.Close 20	0.203330	0.204657	0.994	0.3
L8.Open 63	0.126580	0.172513	0.734	0.4
L8.High 29	-0.123982	0.156606	-0.792	0.4
L8.Low 03	-0.091549	0.136542	-0.670	0.5
L8.Close 39	0.096407	0.205247	0.470	0.6
L9.Open 64	0.029160	0.170513	0.171	0.8
L9.High 35	-0.051521	0.152272	-0.338	0.7
L9.Low 29	-0.065150	0.134824	-0.483	0.6
L9.Close 55	0.036488	0.199576	0.183	0.8
L10.Open 98	-0.000353	0.164337	-0.002	0.9
L10.High 94	0.100213	0.146636	0.683	0.4
L10.Low 11	0.031129	0.129923	0.240	0.8
L10.Close 95	-0.074705	0.190657	-0.392	0.6
L11.Open 61	-0.113984	0.154583	-0.737	0.4
L11.High 35	0.011229	0.138722	0.081	0.9
L11.Low 74	0.019557	0.123317	0.159	0.8
L11.Close 07	-0.091050	0.177176	-0.514	0.6
L12.Open 14	-0.050532	0.137647	-0.367	0.7
L12.High 98	0.132123	0.126862	1.041	0.2
L12.Low	-0.044407	0.112899	-0.393	0.6

```

94
L12.Close      0.034837      0.160081      0.218      0.8
28
L13.Open       -0.113345      0.111217     -1.019      0.3
08
L13.High       0.149498      0.109119      1.370      0.1
71
L13.Low        0.130739      0.095325      1.372      0.1
70
L13.Close      -0.132705      0.134069     -0.990      0.3
22
L14.Open       -0.080742      0.066961     -1.206      0.2
28
L14.High       0.138995      0.082838      1.678      0.0
93
L14.Low        0.060199      0.069125      0.871      0.3
84
L14.Close      -0.134141      0.098855     -1.357      0.1
75

```

```

=====
==

```

Correlation matrix of residuals

	Open	High	Low	Close
Open	1.000000	0.427114	0.509063	0.291196
High	0.427114	1.000000	0.536804	0.826446
Low	0.509063	0.536804	1.000000	0.760544
Close	0.291196	0.826446	0.760544	1.000000

Durbin Watson Test

In [82]:

```

from statsmodels.stats.stattools import durbin_watson
out = durbin_watson(model_fitted.resid)

for col, val in zip(df_stock_var.columns, out):
    print(col, ': ', round(val, 2))

```

```

Open : 2.01
High : 1.94
Low : 2.0
Close : 1.97

```

Getting the Forecast

In [83]:

```
# Get the lag order
lag_order = model_fitted.k_ar
print(lag_order)

# Input data for forecasting
forecast_input = df_differenced.values[-lag_order:]
forecast_input
```

14

Out[83]:

```
array([[ 2.09997559,  0.54998779,  2.20001221,  0.64996338],
       [ 1.70001221, 16.5         ,  2.59997559, 14.45001221],
       [11.65002441,  3.05004883, 12.         ,  2.95001221],
       [ 4.04998779, -4.20001221,  0.29998779, -2.95001221],
       [-1.09997559,  7.45001221,  1.35003662,  5.59997559],
       [-6.85003662, -4.15002441, -4.60003662,  0.65002441],
       [10.95001221,  0.09997559, -4.14996338, -12.79998779],
       [-9.         , -3.39996338,  1.64996338, 10.84997559],
       [ 6.20001221, 10.75         ,  6.04998779,  6.70001221],
       [ 9.         ,  6.04998779, 13.60003662,  9.40002441],
       [ 9.79998779,  0.70001221, -21.35003662, -15.10003662],
       [-16.70001221, -13.75         , -1.29998779, -0.5         ],
       [-0.5         , 14.         ,  8.15002441, 15.20001221],
       [15.15002441, 77.79998779, 15.45001221, 70.95001221]])
```

In [84]:

```
nobs = 14
fc = model_fitted.forecast(y=forecast_input, steps=nobs)
df_forecast = pd.DataFrame(fc, index=df_stock_var.index[-nobs:], columns=df_stock_var.columns + '_1d')
df_forecast
```

Out[84]:

	Open_1d	High_1d	Low_1d	Close_1d
Date				
2021-08-02	74.631703	8.345543	60.785910	-2.779480
2021-08-03	-5.587749	-4.713384	-2.379695	-2.785877
2021-08-04	0.987900	3.992711	2.180547	7.738112
2021-08-05	2.402323	-2.540841	7.151147	0.852943
2021-08-06	1.149127	-0.665813	-4.637790	-3.873159
2021-08-09	-6.732674	-2.508489	-7.163359	-4.079490
2021-08-10	-1.669564	-1.560228	2.959929	1.331051
2021-08-11	3.239984	-0.934133	-2.428122	-2.262858
2021-08-12	-3.408072	-0.184025	0.616127	3.354058
2021-08-13	1.563388	1.984193	-1.194045	-1.812676
2021-08-16	-0.652943	-2.768035	-0.947352	-2.245825
2021-08-17	-1.140102	1.220628	2.354131	4.344306
2021-08-18	2.169831	0.917179	-0.491115	-0.901309
2021-08-20	-1.463496	1.216099	0.059835	0.744216

Inverting forecast back to original scale

In [85]:

```
def invert_transformation(df_train, df_forecast, first_diff=False):
    """Revert back the differencing to get the forecast to original scale."""
    df_fc = df_forecast.copy()
    columns = df_train.columns
    for col in columns:
        if first_diff:
            # Roll back 1st Diff
            df_fc[str(col)+'_forecast'] = df_train[col].iloc[-1] + df_fc[str(col)+'_1d'].
cumsum()
    return df_fc
```

In [86]:

```
df_results = invert_transformation(df_stock_var_train, df_forecast, first_diff=True)
df_results.loc[:, ['Open_forecast', 'High_forecast', 'Low_forecast', 'Close_forecast']]
```

Out[86]:

	Open_forecast	High_forecast	Low_forecast	Close_forecast
Date				
2021-08-02	777.581715	792.095543	756.935935	771.170532
2021-08-03	771.993966	787.382159	754.556240	768.384655
2021-08-04	772.981866	791.374870	756.736787	776.122767
2021-08-05	775.384189	788.834030	763.887934	776.975709
2021-08-06	776.533316	788.168217	759.250144	773.102551
2021-08-09	769.800641	785.659728	752.086785	769.023061
2021-08-10	768.131078	784.099500	755.046714	770.354111
2021-08-11	771.371062	783.165367	752.618592	768.091254
2021-08-12	767.962990	782.981343	753.234719	771.445311
2021-08-13	769.526378	784.965536	752.040674	769.632635
2021-08-16	768.873434	782.197501	751.093322	767.386810
2021-08-17	767.733332	783.418129	753.447453	771.731116
2021-08-18	769.903163	784.335308	752.956338	770.829806
2021-08-20	768.439667	785.551407	753.016172	771.574023

Plotting Forecast vs Actual

In [87]:

```
fig, axes = plt.subplots(nrows=4, ncols=1, dpi=150, figsize=(10,10))
for i, (col,ax) in enumerate(zip(df_stock_var.columns, axes.flatten())):
    df_results[col+'_forecast'].plot(legend=True, ax=ax).autoscale(axis='x',tight=True)
    df_stock_var_test[col][-nobs:].plot(legend=True, ax=ax);
    ax.set_title(col + ": Forecast vs Actuals")
    ax.xaxis.set_ticks_position('none')
    ax.yaxis.set_ticks_position('none')
    ax.spines["top"].set_alpha(0)
    ax.tick_params(labelsize=6)

plt.tight_layout();
```



Evaluating the model

In [88]:

```
from statsmodels.tsa.stattools import acf
def forecast_accuracy(forecast, actual):
    rmse = np.mean((forecast - actual)**2)**.5
    rmse_p = (rmse/np.mean(actual))*100
    return({'rmse':rmse, 'rmse_mean':rmse_p})

print('Forecast Accuracy of Open')
accuracy_prod = forecast_accuracy(df_results['Open_forecast'].values, df_stock_var_test['Open'])
for k, v in accuracy_prod.items():
    print(k, ': ', round(v,4))

print('\nForecast Accuracy of High')
accuracy_prod = forecast_accuracy(df_results['High_forecast'].values, df_stock_var_test['High'])
for k, v in accuracy_prod.items():
    print(k, ': ', round(v,4))

print('\nForecast Accuracy of Low')
accuracy_prod = forecast_accuracy(df_results['Low_forecast'].values, df_stock_var_test['Low'])
for k, v in accuracy_prod.items():
    print(k, ': ', round(v,4))

print('\nForecast Accuracy of Close')
accuracy_prod = forecast_accuracy(df_results['Close_forecast'].values, df_stock_var_test['Close'])
for k, v in accuracy_prod.items():
    print(k, ': ', round(v,4))
```

Forecast Accuracy of Open

rmse : 13.9809

rmse_mean : 1.785

Forecast Accuracy of High

rmse : 8.8607

rmse_mean : 1.1172

Forecast Accuracy of Low

rmse : 20.8148

rmse_mean : 2.6922

Forecast Accuracy of Close

rmse : 14.1482

rmse_mean : 1.8081

Linear Regression

In [195]:

```
df_stock_train_lin=df_stock.loc['2016-01-01':'2021-07-31'].copy()  
df_stock_test_lin=df_stock.loc['2021-07-31:'].copy()
```

Normalize the data

In [196]:

```
from sklearn.preprocessing import MinMaxScaler
```

In [197]:

```
scaler = MinMaxScaler()  
df_stock_train_lin[['Open', 'High', 'Low', 'Volume', 'NIFTY50', 'Range']] = scaler.fit_transform(df_stock_train_lin[['Open', 'High', 'Low', 'Volume', 'NIFTY50', 'Range']])  
df_stock_test_lin[['Open', 'High', 'Low', 'Volume', 'NIFTY50', 'Range']] = scaler.transform(df_stock_test_lin[['Open', 'High', 'Low', 'Volume', 'NIFTY50', 'Range']])
```

In [198]:

```
df_stock_train_lin.head()
```

Out[198]:

	Open	High	Low	Close	Volume	Day of Week	Day of Month	Month	NIFTY50
Date									
2016-01-01	0.866957	0.854721	0.887094	815.549988	0.008332	4	1	1	0.091661
2016-01-04	0.873791	0.860853	0.867169	799.099976	0.025302	0	4	1	0.090919
2016-01-05	0.861543	0.835062	0.861695	800.500000	0.016148	1	5	1	0.086044
2016-01-06	0.857992	0.832897	0.859361	791.950012	0.022635	2	6	1	0.066755
2016-01-07	0.827816	0.820633	0.837641	783.799988	0.021595	3	7	1	0.070447

In [199]:

```
df_stock_test_lin.head()
```

Out[199]:

	Open	High	Low	Close	Volume	Day of Week	Day of Month	Month	NIFTY50
Date									
2021-08-02	0.822490	0.825863	0.811524	775.000000	0.222292	0	2	8	1.023069
2021-08-03	0.802876	0.830282	0.813588	794.799988	0.174073	1	3	8	1.037370
2021-08-04	0.833141	0.818649	0.839526	785.750000	0.066552	2	4	8	1.041369
2021-08-05	0.823378	0.812337	0.845988	786.150024	0.057237	3	5	8	1.035070
2021-08-06	0.825153	0.816034	0.842847	785.200012	0.045588	4	6	8	1.037309

Checking for multicollinearity

In [200]:

```
from statsmodels.stats.outliers_influence import variance_inflation_factor

def calculate_vif(X, thresh=10, verbose=True):
    cols = X.columns
    variables = np.arange(X.shape[1])
    dropped=True
    while dropped:
        dropped=False
        c = X[cols[variables]].values
        vif = [variance_inflation_factor(c, ix) for ix in np.arange(c.shape[1])]

        maxloc = vif.index(max(vif))
        if max(vif) > thresh:
            if verbose == True:
                print('dropping \' + X[cols[variables]].columns[maxloc] + \' at index: ' + str(maxloc))
            variables = np.delete(variables, maxloc)
            dropped=True
        if verbose == True:
            print('Remaining variables:')
            print(X.columns[variables])
    return X[cols[variables]]
```

Model Generation

In [201]:

```
def forward_regression(X, y,
                      threshold_in,
                      verbose=False):
    initial_list = []
    included = list(initial_list)
    while True:
        changed=False
        excluded = list(set(X.columns)-set(included))
        new_pval = pd.Series(index=excluded)
        for new_column in excluded:
            model = sm.OLS(y, sm.add_constant(pd.DataFrame(X[included+[new_column]]))).
fit()
            new_pval[new_column] = model.pvalues[new_column]
        best_pval = new_pval.min()
        if best_pval < threshold_in:
            best_feature = new_pval.idxmin()
            included.append(best_feature)
            changed=True
            if verbose:
                print('Add {:30} with p-value {:.6}'.format(best_feature, best_pval))

        if not changed:
            break

    return included
```

In [202]:

```
def backward_regression(X, y,
                       threshold_out,
                       verbose=False):
    included=list(X.columns)
    while True:
        changed=False
        model = sm.OLS(y, sm.add_constant(pd.DataFrame(X[included]))).fit()
        pvalues = model.pvalues.iloc[1:]
        worst_pval = pvalues.max()
        if worst_pval > threshold_out:
            changed=True
            worst_feature = pvalues.idxmax()
            included.remove(worst_feature)
            if verbose:
                print('Drop {:30} with p-value {:.6}'.format(worst_feature, worst_pval
))
        if not changed:
            break
    return included
```

In [203]:

```

def model_generator(df_train,df_test):
    train_dummy=pd.get_dummies(df_train,columns=['Day of Week','Day of Month','Month'])
    test_dummy = pd.get_dummies(df_test,columns=['Day of Week','Day of Month','Month'])
    for i in [1,2,3,4,5,6,7,9,10,11,12]:
        test_dummy['Month_'+str(i)] = 0
        test_dummy['Month_'+str(i)] = test_dummy['Month_'+str(i)].astype('uint8')
    for i in range(1,32):
        if (i not in df_test['Day of Month'].unique()):
            test_dummy['Day of Month_'+str(i)] = 0
            test_dummy['Day of Month_'+str(i)] = test_dummy['Day of Month_'+str(i)].astype(
'uint8')
    X=train_dummy.drop('Close',axis=1)
    y=train_dummy["Close"]
    X_f=calculate_vif_(X.select_dtypes(exclude='uint8'),10,False)
    X_f=pd.concat([X_f,X.iloc[:,6:]],axis=1)
    model=sm.OLS(y,X_f[backward_regression(X_f,y,0.01)],axis=1).fit()
    pred=model.predict(test_dummy[backward_regression(X_f,y,0.01)])
    rmse = math.sqrt(sklearn.metrics.mean_squared_error(test_dummy['Close'], pred))
    rmse_pct= (rmse/np.mean(test_dummy['Close']))*100
    sig_var = model.pvalues.index.tolist()
    return model,sig_var,rmse_pct

```

In [204]:

```

model,variables,rmse_per = model_generator(df_stock_train_lin,df_stock_test_lin)

```

In [205]:

```
print(model.summary2())
```

Results: Ordinary least squares

```

=====
Model:                OLS                Adj. R-squared:    0.998
Dependent Variable:   Close                AIC:              8558.6303
Date:                2021-09-14 15:27    BIC:              8809.2785
No. Observations:    1369                Log-Likelihood:    -4231.3
Df Model:            47                  F-statistic:       1.622e+04
Df Residuals:        1321                Prob (F-statistic): 0.00
R-squared:           0.998                Scale:            29.354
=====

```

	Coef.	Std.Err.	t	P> t	[0.025	0.975]
High	553.6369	0.6469	855.7805	0.0000	552.3677	554.9060
Range	-40.3435	1.6217	-24.8777	0.0000	-43.5248	-37.1621
Day of Week_0	215.7251	0.3746	575.8248	0.0000	214.9902	216.4601
Day of Week_1	215.9769	0.3690	585.2857	0.0000	215.2530	216.7008
Day of Week_2	216.0589	0.3665	589.5932	0.0000	215.3400	216.7777
Day of Week_3	216.1090	0.3644	593.0256	0.0000	215.3941	216.8239
Day of Week_4	216.6100	0.3756	576.7124	0.0000	215.8732	217.3468
Day of Month_1	35.5627	0.7980	44.5627	0.0000	33.9971	37.1282
Day of Month_2	33.6340	0.8554	39.3188	0.0000	31.9558	35.3121
Day of Month_3	35.7625	0.7874	45.4192	0.0000	34.2178	37.3071
Day of Month_4	35.0445	0.7886	44.4366	0.0000	33.4973	36.5916
Day of Month_5	34.6166	0.7798	44.3937	0.0000	33.0868	36.1463
Day of Month_6	34.3904	0.7871	43.6939	0.0000	32.8463	35.9345
Day of Month_7	34.4036	0.7791	44.1574	0.0000	32.8752	35.9321
Day of Month_8	35.6104	0.7811	45.5894	0.0000	34.0781	37.1428
Day of Month_9	35.8184	0.7808	45.8767	0.0000	34.2868	37.3501
Day of Month_10	34.3149	0.8142	42.1471	0.0000	32.7177	35.9121
Day of Month_11	34.9351	0.7959	43.8930	0.0000	33.3738	36.4965
Day of Month_12	35.2454	0.7889	44.6758	0.0000	33.6977	36.7930
Day of Month_13	35.1836	0.8381	41.9781	0.0000	33.5394	36.8279
Day of Month_14	35.6561	0.8152	43.7370	0.0000	34.0568	37.2554
Day of Month_15	33.1124	0.8061	41.0796	0.0000	31.5311	34.6937
Day of Month_16	35.8237	0.7877	45.4778	0.0000	34.2784	37.3691
Day of Month_17	33.9883	0.7957	42.7169	0.0000	32.4274	35.5492
Day of Month_18	34.9229	0.7792	44.8187	0.0000	33.3943	36.4515
Day of Month_19	34.4477	0.7798	44.1731	0.0000	32.9179	35.9776
Day of Month_20	35.2313	0.7878	44.7221	0.0000	33.6859	36.7767
Day of Month_21	33.3923	0.8156	40.9395	0.0000	31.7922	34.9924
Day of Month_22	35.9373	0.7717	46.5666	0.0000	34.4233	37.4512
Day of Month_23	35.3412	0.7872	44.8966	0.0000	33.7970	36.8855
Day of Month_24	34.4244	0.8042	42.8047	0.0000	32.8467	36.0021
Day of Month_25	35.2365	0.8446	41.7199	0.0000	33.5796	36.8934
Day of Month_26	33.6691	0.8059	41.7801	0.0000	32.0882	35.2500
Day of Month_27	35.6621	0.7713	46.2357	0.0000	34.1490	37.1752
Day of Month_28	33.8734	0.7797	43.4429	0.0000	32.3438	35.4030
Day of Month_29	35.6406	0.8467	42.0948	0.0000	33.9796	37.3016
Day of Month_30	35.8866	0.8352	42.9659	0.0000	34.2481	37.5251
Day of Month_31	33.7118	1.0712	31.4709	0.0000	31.6104	35.8133
Month_1	90.0717	0.4681	192.4303	0.0000	89.1535	90.9900
Month_2	89.6227	0.5080	176.4063	0.0000	88.6260	90.6193
Month_3	90.0961	0.4886	184.4071	0.0000	89.1376	91.0546
Month_4	90.3433	0.5013	180.2340	0.0000	89.3600	91.3267
Month_5	89.8599	0.4767	188.5072	0.0000	88.9248	90.7951
Month_6	90.7097	0.4693	193.2852	0.0000	89.7890	91.6304
Month_7	90.8237	0.4655	195.0900	0.0000	89.9105	91.7370
Month_8	89.7926	0.5181	173.3094	0.0000	88.7762	90.8090
Month_9	89.4868	0.5282	169.4270	0.0000	88.4506	90.5229
Month_10	90.2224	0.5284	170.7422	0.0000	89.1858	91.2590
Month_11	89.2939	0.5276	169.2443	0.0000	88.2588	90.3289

Month_12	90.1570	0.5171	174.3450	0.0000	89.1425	91.1714
----------	---------	--------	----------	--------	---------	---------

Omnibus:	89.140	Durbin-Watson:	2.040
Prob(Omnibus):	0.000	Jarque-Bera (JB):	312.965
Skew:	0.226	Prob(JB):	0.000
Kurtosis:	5.298	Condition No.:	15501098292652602

* The condition number is large ($2e+16$). This might indicate strong multicollinearity or other numerical problems.

In [206]:

```
print("Significance Variables :",'\n',*variables,sep='\n')
```

Significance Variables :

High

Range

Day of Week_0

Day of Week_1

Day of Week_2

Day of Week_3

Day of Week_4

Day of Month_1

Day of Month_2

Day of Month_3

Day of Month_4

Day of Month_5

Day of Month_6

Day of Month_7

Day of Month_8

Day of Month_9

Day of Month_10

Day of Month_11

Day of Month_12

Day of Month_13

Day of Month_14

Day of Month_15

Day of Month_16

Day of Month_17

Day of Month_18

Day of Month_19

Day of Month_20

Day of Month_21

Day of Month_22

Day of Month_23

Day of Month_24

Day of Month_25

Day of Month_26

Day of Month_27

Day of Month_28

Day of Month_29

Day of Month_30

Day of Month_31

Month_1

Month_2

Month_3

Month_4

Month_5

Month_6

Month_7

Month_8

Month_9

Month_10

Month_11

Month_12

In [207]:

```
print("RMSE Percentage",rmse_per,sep='\n')
```

RMSE Percentage
0.5033741107632096

Univariate Forecasting with ARIMA for Linear Regression

In [208]:

```
print("Forecasting Variables :",'\n',list(set(variables) & set(list(df_stock.columns  
))),sep='\n')
```

Forecasting Variables :

['Range', 'High']

In [153]:

```
df_stock_for_test_lin=df_stock_test_lin.iloc[:14].copy()  
dummy_df = pd.DataFrame(index=df_stock_for_test_lin.index)
```

In [154]:

df_stock

Out[154]:

	Open	High	Low	Close	Volume	Day of Week	Day of Month	Month	
Date									
2016-01-01	810.049988	817.900024	806.200012	815.549988	986728.0	4	1	1	77
2016-01-04	813.900024	821.299988	795.099976	799.099976	2388588.0	0	4	1	77
2016-01-05	807.000000	807.000000	792.049988	800.500000	1632394.0	1	5	1	77
2016-01-06	805.000000	805.799988	790.750000	791.950012	2168214.0	2	6	1	75
2016-01-07	788.000000	799.000000	778.650024	783.799988	2082334.0	3	7	1	76
...	...	...	...	...	...	...	...	...	...
2021-08-23	765.000000	770.000000	750.799988	759.349976	2375417.0	0	23	8	166
2021-08-24	759.799988	777.950012	754.250000	771.349976	2732537.0	1	24	8	166
2021-08-25	770.000000	776.000000	759.599976	769.049988	2254852.0	2	25	8	166
2021-08-26	765.150024	771.099976	759.250000	760.549988	1735864.0	3	26	8	167
2021-08-27	760.750000	777.000000	758.099976	771.799988	2432862.0	4	27	8	169

1388 rows × 10 columns



In [174]:

```

Stationary=False
dvalue=0
Stationary_df=df_stock['Low'].copy()
while(Stationary==False):
    dfctest = adfuller(Stationary_df.dropna(),autolag='AIC')
    dfoutput=pd.Series(dfctest[0:4],index=['Test Statistic','p-value','#lags used','number
of observations used'])
    for key,value in dfctest[4].items():
        dfoutput['critical value (%s)'%key]= value
    print(round(dfctest[1]))
    if (dfctest[1]>0.05):
        Stationary=False
        Stationary_df = Stationary_df - Stationary_df.shift(1)
    else :
        Stationary=True
    dvalue+=1
print("Differencing Value : ",dvalue)

```

0

0

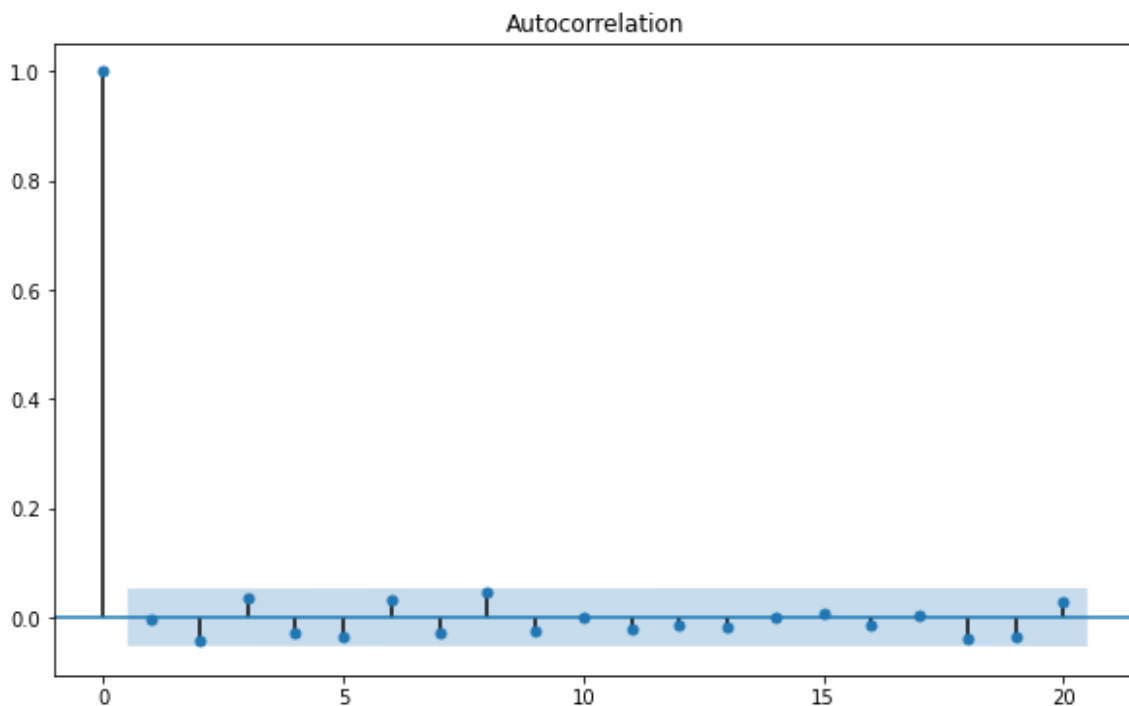
Differencing Value : 2

In [194]:

```

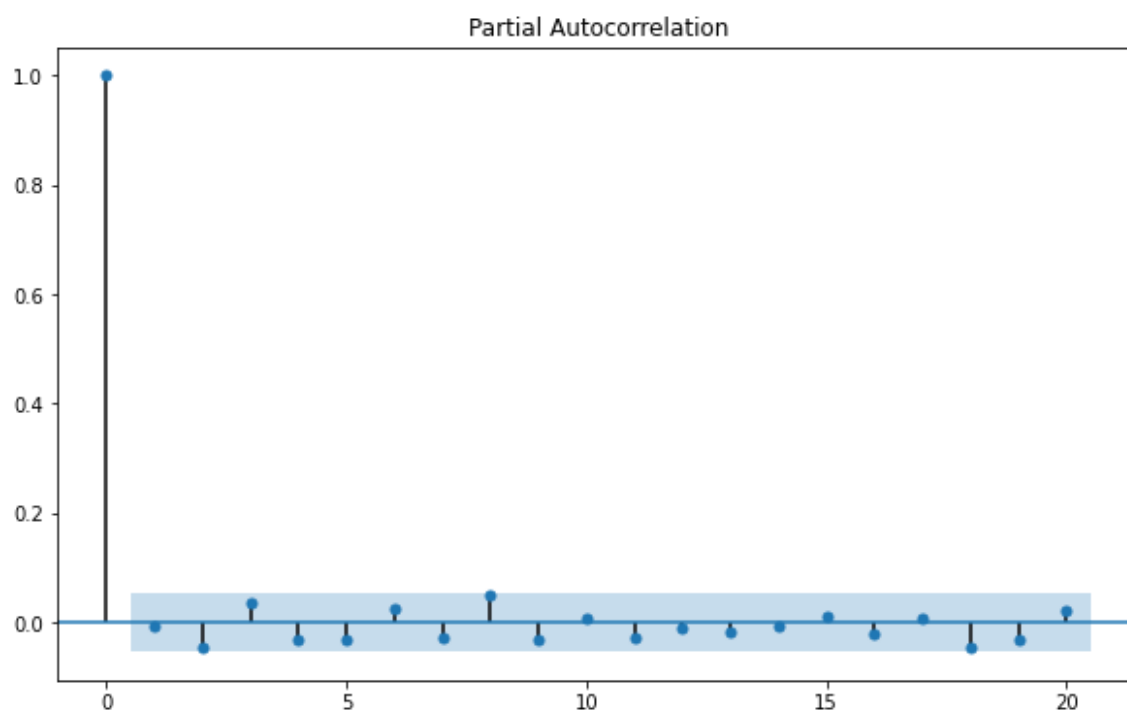
plt.acf(Stationary_df.dropna(),lags=20).show()
plt.show()

```



In [176]:

```
plt.pacf(Stationary_df.dropna(),lags=20).show()  
plt.show()
```



In [177]:

```
df_stock_train=df_stock.loc['2016-01-01':'2021-07-31']['Low']  
df_stock_test=df_stock.loc['2021-07-31:']['Low']
```

In [178]:

```
df_stock_test
```

Out[178]:

```
Date
2021-08-02    764.099976
2021-08-03    765.250000
2021-08-04    779.700012
2021-08-05    783.299988
2021-08-06    781.549988
2021-08-09    782.000000
2021-08-10    782.200012
2021-08-11    773.299988
2021-08-12    773.150024
2021-08-13    767.650024
2021-08-16    771.049988
2021-08-17    778.200012
2021-08-18    779.299988
2021-08-20    743.349976
2021-08-23    750.799988
2021-08-24    754.250000
2021-08-25    759.599976
2021-08-26    759.250000
2021-08-27    758.099976
Name: Low, dtype: float64
```

In [179]:

```
history = [x for x in df_stock_train.values]
predictions = list()
for t in range(len(df_stock_test.values[:14])):
    model = ARIMA(history, order=(0,2,0))
    model_fit = model.fit()
    output = model_fit.forecast()
    yhat = output[0]
    predictions.append(yhat)
    obs = df_stock_test.values[t]
    history.append(obs)
```

In [180]:

```
predictions
```

Out[180]:

```
[711.6000366210949,
 832.0499267577915,
 766.4000244140625,
 794.1500244140626,
 786.8999633789051,
 779.7999877929674,
 782.4500122070318,
 782.4000244140624,
 764.3999633789035,
 773.0000610351556,
 762.1500244140647,
 774.4499511718756,
 785.3500366210935,
 780.3999633789055]
```

In [181]:

```
dummy_df['Low'] = predictions.copy()
dummy_df
```

Out[181]:

	Low	High
Date		
2021-08-02	711.600037	794.692140
2021-08-03	832.049927	800.367594
2021-08-04	766.400024	806.504859
2021-08-05	794.150024	796.734928
2021-08-06	786.899963	795.202650
2021-08-09	779.799988	797.155501
2021-08-10	782.450012	792.918497
2021-08-11	782.400024	800.534536
2021-08-12	764.399963	794.372805
2021-08-13	773.000061	780.086695
2021-08-16	762.150024	790.836510
2021-08-17	774.449951	781.720655
2021-08-18	785.350037	796.135175
2021-08-20	780.399963	790.463020

In [183]:

```
dummy_df['Range'] = dummy_df['High']-dummy_df['Low']
dummy_df.drop(columns=['Low'],inplace=True)
```

In [185]:

```
dummy_df
```

Out[185]:

	High	Range
Date		
2021-08-02	794.692140	83.092103
2021-08-03	800.367594	-31.682333
2021-08-04	806.504859	40.104835
2021-08-05	796.734928	2.584904
2021-08-06	795.202650	8.302687
2021-08-09	797.155501	17.355513
2021-08-10	792.918497	10.468485
2021-08-11	800.534536	18.134512
2021-08-12	794.372805	29.972842
2021-08-13	780.086695	7.086634
2021-08-16	790.836510	28.686486
2021-08-17	781.720655	7.270704
2021-08-18	796.135175	10.785139
2021-08-20	790.463020	10.063057

In [210]:

```
df_stock_for_test_lin[list(set(variables) & set(list(df_stock.columns)))] = dummy_df[list(set(variables) & set(list(df_stock.columns)))].copy()
```

In [211]:

```
print("Forecasted predictors test data")
df_stock_for_test_lin.head()
```

Forecasted predictors test data

Out[211]:

	Open	High	Low	Close	Volume	Day of Week	Day of Month	Month	NIFTY
Date									
2021-08-02	-0.569499	794.692140	-0.558586	775.000000	-0.003612	0	2	8	-0.7784
2021-08-03	-0.569534	800.367594	-0.558583	794.799988	-0.003612	1	3	8	-0.7784
2021-08-04	-0.569481	806.504859	-0.558536	785.750000	-0.003612	2	4	8	-0.7784
2021-08-05	-0.569498	796.734928	-0.558525	786.150024	-0.003612	3	5	8	-0.7784
2021-08-06	-0.569495	795.202650	-0.558530	785.200012	-0.003612	4	6	8	-0.7784

In [212]:

```
df_stock_for_test_lin[['Open', 'High', 'Low', 'Volume', 'NIFTY50', 'Range']] = scaler.trans  
form(df_stock_for_test_lin[['Open', 'High', 'Low', 'Volume', 'NIFTY50', 'Range']])
```

In [213]:

```
model_for, variables_for, rmse_per_for = model_generator(df_stock_train_lin, df_stock_for_  
test_lin)
```

In [214]:

```
print(model_for.summary2())
```

Results: Ordinary least squares

```

=====
Model:                OLS                Adj. R-squared:    0.998
Dependent Variable:   Close                AIC:              8558.6303
Date:                2021-09-14 15:33    BIC:              8809.2785
No. Observations:    1369                Log-Likelihood:    -4231.3
Df Model:             47                  F-statistic:       1.622e+04
Df Residuals:         1321                Prob (F-statistic): 0.00
R-squared:            0.998                Scale:            29.354
=====

```

	Coef.	Std.Err.	t	P> t	[0.025	0.975]
High	553.6369	0.6469	855.7805	0.0000	552.3677	554.9060
Range	-40.3435	1.6217	-24.8777	0.0000	-43.5248	-37.1621
Day of Week_0	215.7251	0.3746	575.8248	0.0000	214.9902	216.4601
Day of Week_1	215.9769	0.3690	585.2857	0.0000	215.2530	216.7008
Day of Week_2	216.0589	0.3665	589.5932	0.0000	215.3400	216.7777
Day of Week_3	216.1090	0.3644	593.0256	0.0000	215.3941	216.8239
Day of Week_4	216.6100	0.3756	576.7124	0.0000	215.8732	217.3468
Day of Month_1	35.5627	0.7980	44.5627	0.0000	33.9971	37.1282
Day of Month_2	33.6340	0.8554	39.3188	0.0000	31.9558	35.3121
Day of Month_3	35.7625	0.7874	45.4192	0.0000	34.2178	37.3071
Day of Month_4	35.0445	0.7886	44.4366	0.0000	33.4973	36.5916
Day of Month_5	34.6166	0.7798	44.3937	0.0000	33.0868	36.1463
Day of Month_6	34.3904	0.7871	43.6939	0.0000	32.8463	35.9345
Day of Month_7	34.4036	0.7791	44.1574	0.0000	32.8752	35.9321
Day of Month_8	35.6104	0.7811	45.5894	0.0000	34.0781	37.1428
Day of Month_9	35.8184	0.7808	45.8767	0.0000	34.2868	37.3501
Day of Month_10	34.3149	0.8142	42.1471	0.0000	32.7177	35.9121
Day of Month_11	34.9351	0.7959	43.8930	0.0000	33.3738	36.4965
Day of Month_12	35.2454	0.7889	44.6758	0.0000	33.6977	36.7930
Day of Month_13	35.1836	0.8381	41.9781	0.0000	33.5394	36.8279
Day of Month_14	35.6561	0.8152	43.7370	0.0000	34.0568	37.2554
Day of Month_15	33.1124	0.8061	41.0796	0.0000	31.5311	34.6937
Day of Month_16	35.8237	0.7877	45.4778	0.0000	34.2784	37.3691
Day of Month_17	33.9883	0.7957	42.7169	0.0000	32.4274	35.5492
Day of Month_18	34.9229	0.7792	44.8187	0.0000	33.3943	36.4515
Day of Month_19	34.4477	0.7798	44.1731	0.0000	32.9179	35.9776
Day of Month_20	35.2313	0.7878	44.7221	0.0000	33.6859	36.7767
Day of Month_21	33.3923	0.8156	40.9395	0.0000	31.7922	34.9924
Day of Month_22	35.9373	0.7717	46.5666	0.0000	34.4233	37.4512
Day of Month_23	35.3412	0.7872	44.8966	0.0000	33.7970	36.8855
Day of Month_24	34.4244	0.8042	42.8047	0.0000	32.8467	36.0021
Day of Month_25	35.2365	0.8446	41.7199	0.0000	33.5796	36.8934
Day of Month_26	33.6691	0.8059	41.7801	0.0000	32.0882	35.2500
Day of Month_27	35.6621	0.7713	46.2357	0.0000	34.1490	37.1752
Day of Month_28	33.8734	0.7797	43.4429	0.0000	32.3438	35.4030
Day of Month_29	35.6406	0.8467	42.0948	0.0000	33.9796	37.3016
Day of Month_30	35.8866	0.8352	42.9659	0.0000	34.2481	37.5251
Day of Month_31	33.7118	1.0712	31.4709	0.0000	31.6104	35.8133
Month_1	90.0717	0.4681	192.4303	0.0000	89.1535	90.9900
Month_2	89.6227	0.5080	176.4063	0.0000	88.6260	90.6193
Month_3	90.0961	0.4886	184.4071	0.0000	89.1376	91.0546
Month_4	90.3433	0.5013	180.2340	0.0000	89.3600	91.3267
Month_5	89.8599	0.4767	188.5072	0.0000	88.9248	90.7951
Month_6	90.7097	0.4693	193.2852	0.0000	89.7890	91.6304
Month_7	90.8237	0.4655	195.0900	0.0000	89.9105	91.7370
Month_8	89.7926	0.5181	173.3094	0.0000	88.7762	90.8090
Month_9	89.4868	0.5282	169.4270	0.0000	88.4506	90.5229
Month_10	90.2224	0.5284	170.7422	0.0000	89.1858	91.2590
Month_11	89.2939	0.5276	169.2443	0.0000	88.2588	90.3289

```
Month_12      90.1570   0.5171 174.3450 0.0000  89.1425  91.1714
-----
Omnibus:      89.140   Durbin-Watson:      2.040
Prob(Omnibus): 0.000   Jarque-Bera (JB):    312.965
Skew:         0.226   Prob(JB):            0.000
Kurtosis:     5.298   Condition No.:       15501098292652602
=====
```

* The condition number is large (2e+16). This might indicate strong multicollinearity or other numerical problems.

In [215]:

```
print("RMSE Percentage for forecasted predictors",rmse_per_for,sep='\n')
```

```
RMSE Percentage for forecasted predictors
1.3902080133248729
```

In [191]: